



Vision-Based Navigation Pipeline for AMRs in Dynamic Warehouse

กระบวนการนำทางด้วยภาพสำหรับหุ่นยนต์เคลื่อนที่อัตโนมัติในคลังสินค้าที่เปลี่ยนแปลงตลอดเวลา

นายบอลตัน อาทิตย์ เดวิส

โครงการนี้เป็นส่วนหนึ่งของการศึกษาตามหลักสูตร
ปริญญาวิศวกรรมศาสตรบัณฑิต สาขาวิชาวิศวกรรมหุ่นยนต์และระบบอัตโนมัติ
สถาบันวิทยาการหุ่นยนต์ภาคสนาม
มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าธนบุรี
ปีการศึกษา 2569

หัวข้อโครงการ	กระบวนการนำทางด้วยภาพสำหรับหุ่นยนต์เคลื่อนที่อัตโนมัติใน คลังสินค้าที่เปลี่ยนแปลงตลอดเวลา
หน่วยกิต	6
ผู้เขียน	นายบอลตัน อาทิตย์ เดวิส
อาจารย์ที่ปรึกษา	ดร. รัตนชัย รมย์ธิติมา
หลักสูตร	วิศวกรรมศาสตรบัณฑิต
สาขาวิชา	วิศวกรรมหุ่นยนต์และระบบอัตโนมัติ
คณะ	สถาบันวิทยาการหุ่นยนต์ภาคสนาม
ปีการศึกษา	2569

บทคัดย่อ

โครงการนี้นำเสนอการพัฒนากระบวนการนำทางด้วยภาพเชิงความหมาย (Semantic Visual SLAM) สำหรับหุ่นยนต์เคลื่อนที่อัตโนมัติ (AMR) เพื่อประยุกต์ใช้ในอุตสาหกรรมคลังสินค้า ปัจจุบันระบบนำทางดั้งเดิมที่อาศัย 2D LiDAR และเครื่องหมาย QR Code มีข้อจำกัดด้านความยืดหยุ่นในการทำงานบนพื้นที่ลาดชันและมีต้นทุนการติดตั้งโครงสร้างพื้นฐานที่สูง นอกจากนี้ ระบบ Visual SLAM มาตรฐานยังมักประสบความล้มเหลวในการระบุตำแหน่งเมื่อเผชิญกับสภาพแวดล้อมพลวัต (Dynamic Environments) ที่มีวัตถุเคลื่อนไหว และเกิดปัญหาหน่วยความจำเกินขีดจำกัดเมื่อต้องสร้างแผนที่อย่างต่อเนื่องเป็นระยะเวลานาน (Lifelong Mapping)

เพื่อแก้ปัญหาดังกล่าว โครงการนี้จึงบูรณาการโครงข่ายประสาทเทียม YOLO26 เข้าเป็นส่วนประมวลผลส่วนหน้าเพื่อทำการตรวจจับและสกัดกั้นจุดเด่นบนวัตถุพลวัต (Dynamic Feature Masking) ออกจากสมการคำนวณเรขาคณิต โดยได้ออกแบบและทดสอบเปรียบเทียบสถาปัตยกรรมระบบนำทาง 2 รูปแบบ ได้แก่ 1) ไปป์ไลน์การประเมินตำแหน่งแบบรวมศูนย์ผ่าน ORB-SLAM3 ร่วมกับการสลับแผนที่พหุคูณ (Atlas Multi-map) และ 2) ไปป์ไลน์แบบแยกส่วนภาระงานที่ใช้ VINS ในการประเมินพิกัดความถี่สูง ผสานกับ RTAB-Map เพื่อบริหารจัดการโหนดแผนที่ผ่านกลไกจำกัดเพดานหน่วยความจำ (Memory Bounded) ระบบทั้งหมดได้รับการออกแบบให้ประมวลผลแบบเรียลไทม์บนคอมพิวเตอร์ระดับเอดจ์ NRU-51V โดยรับข้อมูลความหน่วงต่ำจากกล้องสเตอริโอแบบกว้างพิเศษและหน่วยวัดความเฉื่อย (IMU)

การประเมินประสิทธิภาพดำเนินการผ่านการทดสอบออนไลน์บนหุ่นยนต์จริงในพื้นที่จำลองคลังสินค้า ภายใต้เงื่อนไขตัวแปรความพลวัตจากการสุ่มตำแหน่งสิ่งกีดขวางและสถานะแสงที่แตกต่าง

กัน โดยประเมินผลเชิงปริมาณจากตัวชี้วัดด้านความแม่นยำทางภาพ (mAP, IoU) และความคลาดเคลื่อนของเส้นทางนำทาง (ATE, RPE) เทียบกับระบบอ้างอิงความแม่นยำสูง (Motion Capture) ผลลัพธ์จากโครงการนี้นำไปสู่สถาปัตยกรรมระบบนำทางที่มีความทนทานต่อความเปลี่ยนแปลงของพื้นที่ สามารถทำงานร่วมกับระบบ ROS 2 Nav2 และ Costmap 2D ได้อย่างสมบูรณ์ ซึ่งช่วยยกระดับขีดความสามารถของหุ่นยนต์ AMR และลดต้นทุนการจัดการโครงสร้างพื้นฐานของคลังสินค้าได้อย่างมีประสิทธิภาพ

คำสำคัญ : หุ่นยนต์เคลื่อนที่อัตโนมัติ (AMR), การนำทางด้วยภาพเชิงความหมาย (Semantic Visual SLAM), สภาพแวดล้อมพลวัต (Dynamic Environment), การตรวจจับวัตถุ (Object Detection), การสร้างแผนที่ระยะยาว (Lifelong Mapping)

บทที่ 1 บทนำ

1.1 ความสำคัญและที่มาของงานวิจัย

หุ่นยนต์เคลื่อนที่อัตโนมัติ (Autonomous Mobile Robots: AMRs) มีบทบาทสำคัญในการเพิ่มประสิทธิภาพระบบโลจิสติกส์ในคลังสินค้า ปัจจุบัน บริษัท Gensurv Robotics อาศัยเทคโนโลยีเซนเซอร์ 2D LiDAR ร่วมกับการนำทางด้วยคิวอาร์โค้ด (QR-code) เป็นหลัก อย่างไรก็ตาม โซลูชันดังกล่าวยังคงมีข้อจำกัดทางธุรกิจ ทั้งต้นทุนของเซนเซอร์ LiDAR ที่มีราคาสูง และความยืดหยุ่นที่ต่ำเนื่องจากบริษัทต้องสูญเสียต้นทุนและเวลาในการปรับแต่งโครงสร้างพื้นฐานของคลังสินค้า (Infrastructure modification) เพื่อติดตั้งจุดอ้างอิงเทียม ด้วยเหตุนี้ การเปลี่ยนผ่านเทคโนโลยีจาก LiDAR ไปสู่ ระบบนำทางด้วยภาพ (Vision-based Navigation) ที่ใช้เซนเซอร์กล้องซึ่งมีต้นทุนต่ำกว่า และให้ข้อมูลสภาพแวดล้อมที่สมบูรณ์กว่า จึงเป็นทางเลือกที่ตอบโจทย์ในการลดต้นทุนของบริษัท

อย่างไรก็ตาม การนำเทคโนโลยีกล้องและการสร้างแผนที่พร้อมระบุตำแหน่ง (Visual SLAM) มาใช้งานจริงในคลังสินค้านั้นมีความท้าทายทางวิชาการที่สำคัญ เนื่องจากอัลกอริทึม Visual SLAM มาตรฐานส่วนใหญ่ถูกพัฒนาขึ้นภายใต้สมมติฐานที่ว่า "สภาพแวดล้อมจะต้องอยู่นิ่ง (Static environment)" อ้างอิงจากงานวิจัยของ Saputra et al. (2018) ระบุไว้อย่างชัดเจนว่า เมื่อนำระบบ SLAM ไปใช้งานในสภาพแวดล้อมแบบพลวัต (Dynamic Environments) เช่น คลังสินค้าที่มีการเคลื่อนย้ายพัสดุหรือสิ่งกีดขวางอยู่ตลอดเวลา ความแม่นยำของระบบจะลดลงอย่างรุนแรง เนื่องจากระบบจะดึงคุณลักษณะเด่น (Features) ของวัตถุที่เคลื่อนไหวไปคำนวณตำแหน่งผิดพลาด นอกจากนี้ Cadena et al. (2016) ได้เน้นย้ำว่าความท้าทายสูงสุดของระบบ SLAM ในยุคปัจจุบันคือ "ความทนทาน (Robustness)" และ "การสร้างแผนที่ต่อเนื่อง (Lifelong Mapping)" ที่หุ่นยนต์ต้องสามารถแยกแยะโครงสร้างถาวรออกจากวัตถุพลวัตได้

เพื่อแก้ปัญหาระบบนำทางในสภาพแวดล้อมพลวัต โครงการนี้จึงนำเสนอ การพัฒนากระบวนการนำทางด้วยภาพ (Vision-based Navigation Pipeline) โดยมุ่งเน้นการบูรณาการโมเดลปัญญาประดิษฐ์ YOLO26 เพื่อตรวจจับและคัดกรองวัตถุพลวัตออกจากระบบประเมินตำแหน่ง โดยโครงการจะทำการพัฒนาและเปรียบเทียบประสิทธิภาพของสถาปัตยกรรมระบบนำทาง 2 รูปแบบ ได้แก่ 1) การทำงานร่วมกับกรอบการทำงาน ORB-SLAM3 และ 2) การทำงานร่วมกับระบบ VINS ผสานกับ RTAB-Map เพื่อค้นหาแนวทางที่มีความทนทานสูงสุดสำหรับคลังสินค้า

นอกจากนี้ การเปลี่ยนเซนเซอร์จากการใช้ LiDAR (Active sensor) มาเป็นกล้อง (Passive sensor) อาจส่งผลให้ระบบมีความอ่อนไหวต่อสภาพแสงสว่างในอาคาร โครงการนี้จึงได้ออกแบบการทดลองเพื่อจำลองสภาพแวดล้อมและตัวแปรด้านความเข้มแสงในแต่ละช่วงเวลา เพื่อเก็บข้อมูลและประเมินผลเชิงประจักษ์ว่า ปัจจัยด้านแสงสว่างในคลังสินค้าส่งผลกระทบต่อประสิทธิภาพของระบบ Visual SLAM อย่างมีนัยสำคัญหรือไม่ ซึ่งจะเป็นข้อมูลสำคัญในการพัฒนาและปรับปรุงระบบนำทางของบริษัทในระยะยาวต่อไป

1.2 วัตถุประสงค์ของงานวิจัย

1. พัฒนาระบบระบุตำแหน่งแบบ Visual-Inertial Odometry พร้อมระบบ Lifelong Mapping เพื่ออัปเดตแผนที่คลังสินค้าแบบ Real-time
2. พัฒนาระบบ image enhancement ในสภาวะแวดล้อมที่มีปริมาณแสงต่ำได้ด้วยโมเดล EnlightenGAN
3. พัฒนาระบบรับรู้สิ่งกีดขวางหรือวัตถุเคลื่อนที่ (Dynamic object) จากการทำ Object detection ด้วย Deep learning
4. พัฒนาระบบ Lane Detection ด้วย PolyNET และ Lane tracking ด้วย PID controller
5. Integrated system ต้องสามารถสร้าง 2D cost map และระบุตำแหน่งหุ่นยนต์ได้โดยมีความคลาดเคลื่อนน้อยกว่า 10 เซนติเมตร ในสภาพแวดล้อมที่มีความเข้มแสงแตกต่างกันได้

1.3 ประโยชน์และผลที่คาดว่าจะได้รับจากงานวิจัย

1. ช่วยบริษัทลดต้นทุนที่เกิดจากการปรับเปลี่ยนโครงสร้างพื้นฐานของคลังสินค้าสำหรับระบบนำทางหุ่นยนต์ AMR
2. ได้ Navigation Pipeline ที่ Robust ต่อ Dynamic environment
3. สามารถนำ Pipeline นี้ไป Plugin กับระบบ Control และ Motion planning เพื่อใช้งานจริงบนหุ่นยนต์คลังสินค้าต่อไป

1.4 ขอบเขตงานวิจัย

1. ทำการทดสอบหลักผ่านหุ่นยนต์ Mobile robot ขนาดเล็กที่ประมวลผลด้วยหน่วยประมวลผล NRU-51V และใช้กล้องรุ่น AC-IMX390-H190) เท่านั้น
2. ใน AMR navigation system นั้น ผู้จัดทำได้วางกรอบการทำงานไว้ในส่วนของ Visual SLAM, Object prediction และ Lane tracking controller (กรอบสีแดง) ดังรูปที่ 1 เท่านั้น

3. ทดสอบประสิทธิภาพของ Integrated system ของหุ่นยนต์ในสนามจำลอง Warehouse ที่มีการเปลี่ยนแปลงของพัสดุจำลอง, มีทางเดินที่มีความชัน 5 องศา, มีพัสดุ 5 กอง กองละ 4 กล่อง, มี Starting Point บริเวณขวาล่างที่เป็นสี่เหลี่ยมสีเทา ดังรูปที่ 2 เท่านั้น โดยตำแหน่งของคลังสินค้าจำลอง จะมีการเปลี่ยนแปลงระหว่างการทดสอบเพื่อจำลองสภาพแวดล้อมแบบ Dynamic

1.5 ขั้นตอนการดำเนินงาน

แผนดำเนินงาน	2569						
	มี.ย.	ก.ค.	ส.ค.	ก.ย.	ต.ค.	พ.ย.	ธ.ค.
1. ศึกษาแนวคิดและทฤษฎีที่เกี่ยวข้องกับระบบการนำทางด้วยภาพ (Vision-Based Navigation)							
2. สืบค้นและวิเคราะห์ข้อมูลเกี่ยวกับอัลกอริทึม Visual SLAM, ระบบปรับปรุงคุณภาพภาพในสภาวะแสงน้อย (Image Enhancement: EnlightenGAN), ระบบตรวจจับวัตถุ (Object Detection: YOLO26)							
3. ออกแบบโครงสร้างและไปป์ไลน์การทำงานร่วมกันระหว่างระบบฮาร์ดแวร์และซอฟต์แวร์ (Software Pipeline)							
4. กำหนดคุณสมบัติและการเชื่อมต่อของเซนเซอร์กล้อง AC-IMX390-H190 ร่วมกับหน่วยประมวลผลระดับเอดจ์ (Edge Computing) รุ่น NRU-51V บนตัวหุ่นยนต์ Mobile Robot							
6. บูรณาการระบบย่อยทั้งหมดเข้าด้วยกันภายใต้กรอบการทำงานระบบนำทางของหุ่นยนต์ AMR							
7. จัดเตรียมพื้นที่คลังสินค้าจำลอง (Zone A และ Zone B) ที่มีความลาดชัน 5 องศา และจัดวางพัสดุจำลองเพื่อสร้างสภาพแวดล้อมแบบพลวัต (Dynamic Environment)							
8. ดำเนินการทดสอบระบบแบบบูรณาการ (Integrated System Experiment / Live Test) โดยให้หุ่นยนต์เคลื่อนที่ด้วยความเร็ว 1.5 เมตรต่อวินาที ภายใต้เงื่อนไขการเปลี่ยนแปลงของแสงสว่างตามช่วงเวลาที่กำหนด							
9. รวบรวมข้อมูลจากการทดสอบมาประเมินผลประสิทธิภาพของระบบตามตัวชี้วัดที่กำหนด เช่น ค่าความคลาดเคลื่อนของเส้นทาง (ATE, RPE) ความหน่วงของระบบ (Latency) และความแม่นยำในการตรวจจับวัตถุ (IoU, mAP)							
10. สรุปผลการดำเนินงาน อภิปรายผล และจัดทำเล่มรายงานโครงการวิศวกรรมฉบับสมบูรณ์							

รูปที่ Gantt chart

บทที่ 2 ทฤษฎี/งานวิจัยที่เกี่ยวข้อง

การดำเนินโครงการ "กระบวนการนำทางด้วยภาพสำหรับหุ่นยนต์เคลื่อนที่อัตโนมัติในคลังสินค้าที่เปลี่ยนแปลงตลอดเวลา" (Vision-Based Navigation Pipeline for AMRs in Dynamic Warehouse) คณะผู้จัดทำมีเป้าหมายในการพัฒนาและทดสอบประสิทธิภาพเฉพาะในส่วนของไปป์ไลน์ระบบนำทาง (Navigation Pipeline) อ้างอิงตามขอบเขตการศึกษาที่ได้กำหนดไว้เท่านั้น โครงการนี้ไม่ได้มุ่งเน้นการพัฒนา ระบบตัวหุ่นยนต์อัตโนมัติทั้งหมด (Full Autonomous System) ขึ้นมาใหม่ แต่เป็นการบูรณาการซอฟต์แวร์นำทางเข้ากับสถาปัตยกรรมเดิมของหุ่นยนต์เคลื่อนที่อัตโนมัติ (AMR) ที่มีอยู่แล้ว เพื่อให้การพัฒนา ระบบนำทางดังกล่าวเป็นไปอย่างมีประสิทธิภาพ คณะผู้จัดทำจึงได้ทำการศึกษา ค้นคว้า และรวบรวมทฤษฎี เอกสาร และงานวิจัยที่เกี่ยวข้องในหัวข้อหลักดังต่อไปนี้

2.1 ระบบนำทางของหุ่นยนต์เคลื่อนที่อัตโนมัติ (AMR Navigation Systems)

หุ่นยนต์ AMR (Autonomous Mobile Robotic) เป็นหนึ่งในเทคโนโลยีที่กำลังเปลี่ยนแปลงอุตสาหกรรมในยุคดิจิทัล โดยเฉพาะในระบบการผลิต คลังสินค้า และการจัดการวัสดุ ด้วยความสามารถในการเรียนรู้ ความยืดหยุ่น และความแม่นยำที่สูง หุ่นยนต์ AMR ช่วยลดต้นทุนและเพิ่มประสิทธิภาพได้อย่างก้าวกระโดด บทความนี้จะลงลึกถึงคุณสมบัติ เทคโนโลยี และกรณีการใช้งานของหุ่นยนต์ AMR เพื่อช่วยให้คุณเข้าใจถึงศักยภาพของนวัตกรรมนี้มากยิ่งขึ้น

2.1.1 การจัดการโลจิสติกส์และปัญหาคอขวดในอดีต

ในสภาพแวดล้อมของคลังสินค้าแบบดั้งเดิม ค่าใช้จ่ายที่สูงและสูญเปล่าส่วนใหญ่มักเกิดจากกระบวนการโลจิสติกส์ในการเคลื่อนย้ายวัสดุจากจุดหนึ่งไปยังอีกจุดหนึ่ง ซึ่งเป็นงานที่สร้างความเหนื่อยล้าให้กับพนักงานที่ต้องเดินเท้าพร้อมกับแบกรับน้ำหนักอย่างต่อเนื่อง การเคลื่อนย้ายเหล่านี้ก่อให้เกิด "ความสูญเปล่าจากการเคลื่อนไหว" (Motion waste) ซึ่งเป็นหนึ่งใน 8 ความสูญเปล่าหลักตามหลักการผลิตแบบลีน (Lean manufacturing) เพื่อรักษาขีดความสามารถในการแข่งขันและลดต้นทุนด้านโลจิสติกส์ คลังสินค้าอัจฉริยะ (Smart warehouses) จึงเริ่มนำหุ่นยนต์อัตโนมัติเข้ามาทำงานร่วมกับมนุษย์เพื่อลดระยะเวลาและความสูญเปล่าดังกล่าวนั่นเอง

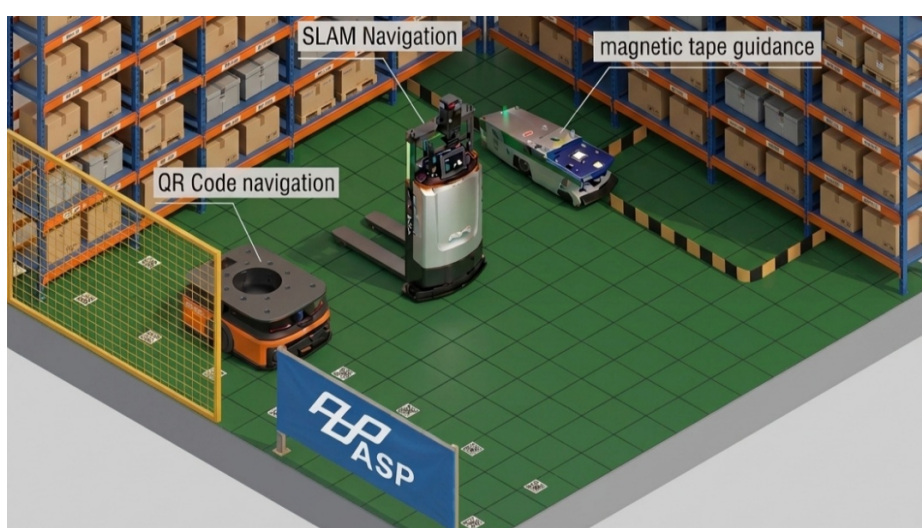


รูปที่ 1 ความสูญเปล่าหลักตามหลักการผลิตแบบลีน (Lean manufacturing) 8 ประการ

แหล่งที่มา : <https://teachme-biz.com/th/blog/reduce-waste/>

2.1.2 ยุกของรอนำทางอัตโนมัติ (Automated Guided Vehicles: AGVs)

การวิจัยและพัฒนาหุ่นยนต์เคลื่อนที่อัตโนมัติเริ่มต้นขึ้นตั้งแต่ช่วงทศวรรษที่ 1950 โดยเริ่มต้นจากการใช้งานรอนำทางอัตโนมัติ หรือ AGV อย่างไรก็ตาม ระบบของ AGV มีข้อจำกัดที่สำคัญคือ ตัวหุ่นยนต์จำเป็นต้องพึ่งพาโครงสร้างพื้นฐานที่ถูกติดตั้งไว้ล่วงหน้าในคลังสินค้า (Previously installed infrastructure) ในการนำทาง เช่น การเดินเส้นแถบแม่เหล็ก คิวอาร์โค้ด หรือการใช้จุดอ้างอิงเทียม (Artificial landmarks) นอกจากนี้ AGV ยังต้องเคลื่อนที่ตามเส้นทางที่ถูกวางแผนไว้ล่วงหน้าอย่างตายตัว (Pre-planned routes) เท่านั้น ทำให้ระบบขาดความยืดหยุ่นทางพื้นที่ และหากสภาพแวดล้อมในคลังสินค้านี้มีการเปลี่ยนแปลง AGV จะไม่สามารถปรับตัวได้ทันที



รูปที่ 2 แสดงหุ่นยนต์ในคลังสินค้า 3 ตัวที่มีเทคโนโลยีการนำทางต่างกัน ได้แก่ หุ่นยนต์ที่ใช้การนำทางด้วยรหัส QR, หุ่นยนต์ที่กวาดเลเซอร์กลางใช้การนำทางแบบ SLAM โดยไม่ใช้โครงสร้างพื้นฐานบนพื้น และหุ่นยนต์สีขาวใช้การนำทางด้วยแถบแม่เหล็ก

แหล่งที่มา : <https://www.linkedin.com/pulse/agv-amr-navigation-technologies-which-one-drives-your-abdul-rahman-6pjyc>

2.1.3 การเปลี่ยนผ่านหุ่นยนต์เคลื่อนที่อัตโนมัติ (Autonomous Mobile Robots: AMRs)

เพื่อแก้ไขข้อจำกัดของระบบ AGV จึงได้มีการพัฒนาหุ่นยนต์ AMR ขึ้นมา โดยความแตกต่างที่สำคัญคือ AMR มีขีดความสามารถในการตัดสินใจด้วยตนเอง และไม่จำเป็นต้องพึ่งพาโครงสร้างพื้นฐานภายนอกที่ติดตั้งไว้ล่วงหน้าเพื่อใช้ในการนำทาง ความก้าวหน้าด้านฮาร์ดแวร์ในช่วงที่ผ่านมา โดยเฉพาะเซนเซอร์รับภาพ (เช่น กล้อง 2D/3D และ LiDAR) ประกอบกับหน่วยประมวลผลประสิทธิภาพสูง ทำให้ AMR สามารถรับรู้และประเมินสภาพแวดล้อมรอบตัวได้อย่างแม่นยำ คุณสมบัติเด่นที่ทำให้ AMR เข้ามาแทนที่ AGV คือขีดความสามารถในการทำงานภายใต้สภาพแวดล้อมแบบพลวัต (Dynamic environment) เมื่อ AMR ตรวจพบสิ่งกีดขวางในเส้นทาง ระบบจะสามารถประมวลผลเพื่อค้นหาและวางแผนเส้นทางอื่น (Alternate routes) เพื่อเคลื่อนที่หลบหลีกสิ่งกีดขวางหรือหุ่นยนต์ตัวอื่นได้โดยอัตโนมัติ โดยที่ไม่จำเป็นต้องให้ผู้ใช้งานเข้ามาออกแบบเส้นทางใหม่ให้

ด้วยความสามารถในการตัดสินใจและนำทางได้อย่างอิสระ ปัจจุบันภาคอุตสาหกรรมขนาดใหญ่จึงเปลี่ยนมาใช้ AMR กันอย่างแพร่หลาย ตัวอย่างเช่น การประยุกต์ใช้ยานพาหนะขนส่งไร้คนขับแบบอัตโนมัติ (Autonomous driverless transport vehicle) ภายในโรงงานอัดขึ้นรูปโลหะ (Press plant) ของ BMW Group ที่เมือง Regensburg (อ้างอิง: BMW Group Press, 2024) ซึ่งระบบของยานพาหนะสามารถนำทางและเคลื่อนที่หลบหลีกผ่านพื้นที่ต่างๆ ในโรงงานจริงที่มีความพลุกพล่านและซับซ้อนได้อย่างแม่นยำ (Precise navigation) แสดงให้เห็นถึงประสิทธิภาพของ AMR ที่ก้าวข้ามข้อจำกัดของระบบนำทางแบบเดิมได้อย่างสมบูรณ์



รูปที่ 3 รถขนส่งอัตโนมัติไร้คนขับนำทางได้อย่างแม่นยำภายในโรงงานปั๊มขึ้นรูปของ BMW Group สาขาเรเกนสเบิร์ก (Regensburg)

แหล่งที่มา : <https://www.press.bmwgroup.com/global/video/detail/PF0009726/autonomous-driverless-transport-vehicle-navigates-precisely-through-bmw-group-plant-regensburg-press-plant>

2.2 เทคโนโลยีการระบุตำแหน่งและการสร้างแผนที่พร้อมกันด้วยภาพ (Visual SLAM)

ในการทำงานของหุ่นยนต์เคลื่อนที่อัตโนมัติ (AMR) ระบบการระบุตำแหน่ง (Localization) ถือเป็นหัวใจสำคัญที่ทำให้หุ่นยนต์สามารถเคลื่อนที่ได้อย่างแม่นยำ โดยวิธีการระบุตำแหน่งที่ได้รับการยอมรับและใช้งานอย่างแพร่หลายคือเทคโนโลยี SLAM (Simultaneous Localization and Mapping) ซึ่งระบบนำทางแบบ SLAM จะทำการดึงข้อมูลจุดเด่น (Features) ที่อยู่ในสภาพแวดล้อมมาใช้ในการสร้างแผนที่ ซอฟต์แวร์ของระบบจะรวบรวมข้อมูลจากตำแหน่งปัจจุบันและตำแหน่งที่หุ่นยนต์เคยผ่านมา เพื่อนำมาประเมินและสร้างการกระจายความน่าจะเป็น (Probability distribution) สำหรับคาดการณ์ตำแหน่งที่ถูกต้องที่สุดของหุ่นยนต์

2.2.1 องค์ประกอบของระบบ Visual SLAM

โครงการนี้มุ่งเน้นการประยุกต์ใช้ Visual SLAM หรือการสร้างแผนที่และระบุตำแหน่งด้วยเซนเซอร์ภาพ เพื่อลดต้นทุนจากเซนเซอร์ LiDAR โดยสถาปัตยกรรมของระบบ Visual SLAM (อ้างอิงจากแผนผังการทำงานของโครงการ) ประกอบด้วย 3 ส่วนหลัก ได้แก่:

1) ระบบส่วนหน้า (SLAM Front-end / Visual Odometry) :

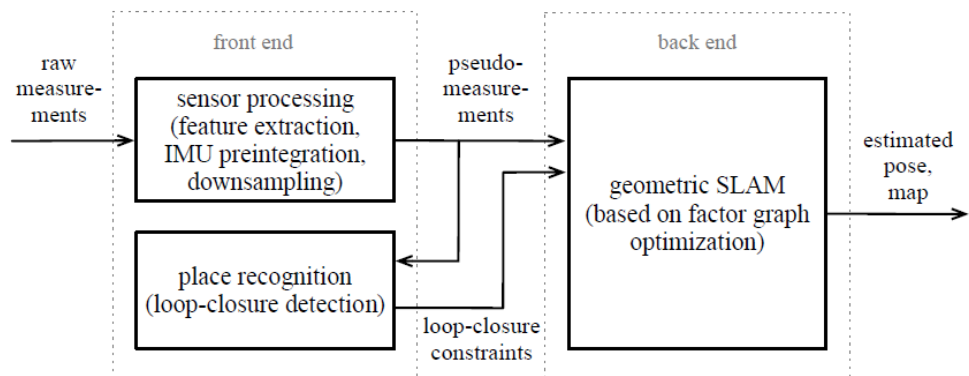
ทำหน้าที่รับข้อมูลภาพจากกล้องและสกัดจุดเด่น (Feature Extraction) เพื่อประเมินการเคลื่อนที่และตำแหน่งของหุ่นยนต์แบบสัมพัทธ์ในระยะเวลาสั้นๆ

2) ระบบส่วนหลัง (SLAM Back-end) :

ทำหน้าที่ปรับปรุงและแก้ไขความคลาดเคลื่อนสะสม (Drift error) ที่เกิดจากการประเมินตำแหน่งของส่วนหน้า โดยอาศัยหลักการปรับแต่งโครงข่ายกราฟ (Graph Optimization) เพื่อให้ได้แผนที่และตำแหน่งที่มีความแม่นยำสูงสุด

3) การตรวจจบวงวนปิด (Loop Closure):

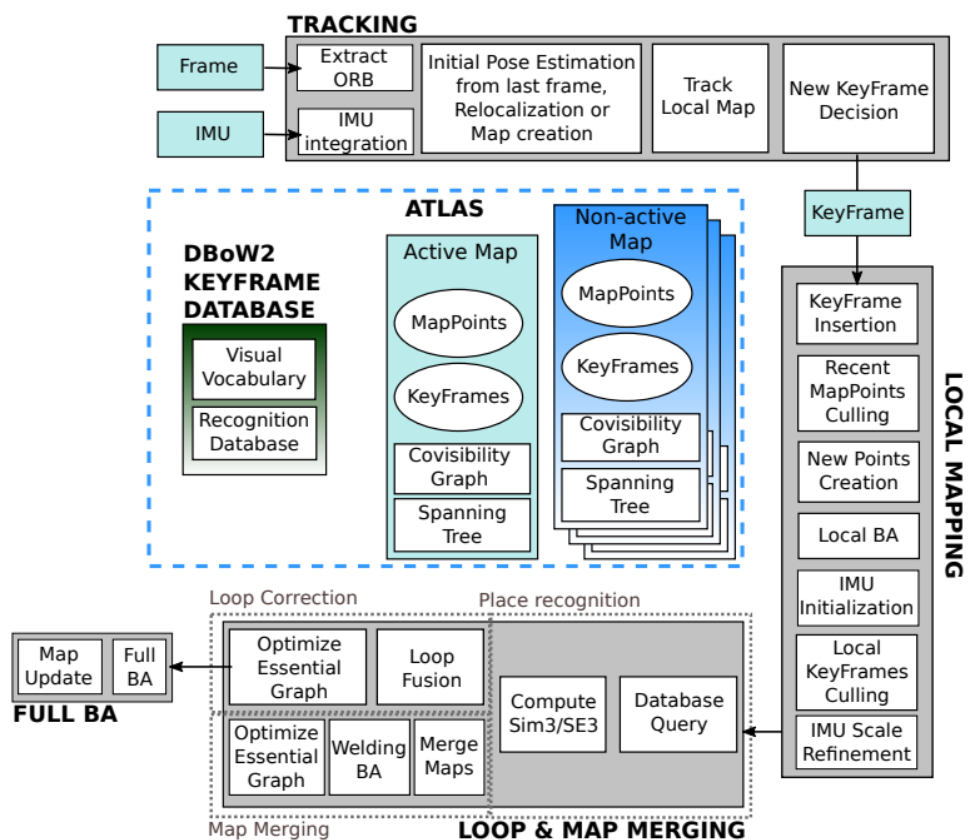
เป็นกระบวนการที่ระบบจะจดจำสถานที่ที่หุ่นยนต์เคยเคลื่อนที่ผ่านมาแล้ว หากหุ่นยนต์วนกลับมาที่เดิม ระบบจะทำการจับคู่ภาพปัจจุบันกับภาพในอดีต และปรับแก้แผนที่ทั้งหมดให้มีความสมบูรณ์และลดความคลาดเคลื่อนสะสม



รูปที่ 4 วิธีการทำงานแบบอ้อม (Indirect methods) ที่พบได้ทั่วไปในระบบ SLAM

2.2.2 ORB-SLAM3

ORB-SLAM3 เป็นระบบ SLAM แบบโอเพนซอร์ส (Open-source) ที่พัฒนาต่อมาจากความสำเร็จของ ORB-SLAM และ ORB-SLAM2 โดยถือเป็นกรอบการทำงาน (Framework) แรกที่สามารถรองรับการทำงานแบบ Visual, Visual-Inertial และ Multi-map SLAM ร่วมกับกล้องได้หลากหลายประเภท ทั้งกล้องเลนส์เดี่ยว (Monocular) กล้องสเตอริโอ (Stereo) และกล้องวัดความลึก (RGB-D) รวมถึงรองรับทั้งแบบเลนส์ปกติ (Pinhole model) และเลนส์ตาปลา (Fisheye model) โดยสถาบันกรรมเชิงลึกและกลไกการทำงานของ ORB-SLAM3 ที่เกี่ยวข้องกับโครงงานนี้



รูปที่ 5 องค์ประกอบหลักในระบบ ORB-SLAM3

1) การผสานข้อมูลภาพและเซนเซอร์วัดความเฉื่อยแบบแนบแน่น (Tightly-coupled Visual-Inertial SLAM):

ความก้าวหน้าที่สำคัญที่สุดประการหนึ่งของ ORB-SLAM3 คือระบบ Visual-Inertial SLAM (VI-SLAM) ที่ใช้วิธีการผสานข้อมูลแบบแนบแน่น (Tightly-integrated) โดยอาศัยหลักการประมาณค่าความน่าจะเป็นแบบ Maximum-a-Posteriori (MAP) ทั้งในขั้นตอนการเริ่มต้นระบบ (Initialization) และการทำงานปกติ ต่างจากระบบอื่นในอดีตที่มักแยกส่วนการคำนวณ ซึ่งมีกลไกย่อยดังนี้:

- **กระบวนการเริ่มต้นระบบความเฉื่อยอย่างรวดเร็ว (Fast IMU Initialization):** ปัญหาหลักของระบบ VI-SLAM ทั่วไปคือความล่าช้าในการคำนวณค่าพารามิเตอร์เริ่มต้นของ IMU แต่ ORB-SLAM3 ได้นำเสนอกระบวนการคำนวณค่าความไม่แน่นอนแบบ MAP ที่สามารถหาค่าสเกลของแผนที่ (Scale), ทิศทางของแรงโน้มถ่วง (Gravity vector), ค่าความคลาดเคลื่อนของไจโรสโคปและมาตรวัดความเร่ง (Gyroscope and Accelerometer biases) ได้อย่างรวดเร็วภายในเวลาไม่เกิน 2 วินาทีหลังจากระบบเริ่มทำงาน ส่งผลให้หุ่นยนต์สามารถระบุตำแหน่งที่ถูกต้องและมีสเกลระยะทางที่ตรงกับความเป็นจริงทางกายภาพได้ทันที
- **การหาค่าพารามิเตอร์ที่เหมาะสมที่สุดผ่านกรอบเวลา (Joint Local Bundle Adjustment):** ในระหว่างการเคลื่อนที่ ระบบจะทำการคำนวณเพื่อหาค่าตำแหน่งและทิศทาง (Pose) ของหุ่นยนต์ ควบคู่ไปกับตำแหน่งของจุดเด่นในสามมิติ (3D Landmarks) และสถานะของ IMU (Velocity และ Biases) พร้อมๆ กัน ภายในพื้นที่กรอบเวลาส่วนท้องถิ่น (Local window) ทำให้ระบบสามารถชดเชยค่าความคลาดเคลื่อนสะสม (Drift) จากภาพและเซนเซอร์ความเฉื่อยได้อย่างมีประสิทธิภาพ

2) ระบบการจัดการแผนที่พหุคูณและการรู้จำสถานที่แบบแอตลาส (Atlas: Multi-Map System):

เพื่อสนับสนุนแนวคิดการสร้างแผนที่แบบต่อเนื่องในระยะยาว (Lifelong Mapping) ภายใต้สภาพแวดล้อมที่มีการเปลี่ยนแปลงหรือมีการรบกวนทางสายตาอย่างคลั่งคลั่ง (Dynamic Warehouse) ORB-SLAM3 ได้นำสถาปัตยกรรมการจัดการแผนที่ที่เรียกว่า Atlas เข้ามาใช้งาน ซึ่งประกอบด้วยแผนที่ 2 สถานะ ได้แก่:

- **Active Map:** แผนที่ย่อยส่วนท้องถิ่นที่หุ่นยนต์กำลังใช้ในการติดตามตำแหน่งและการสร้างแผนที่อยู่ ณ ปัจจุบัน
- **Non-active Maps:** กลุ่มของแผนที่ย่อยในอดีตที่ระบบทำการบันทึกและจัดเก็บเอาไว้ในหน่วยความจำ

เมื่อหุ่นยนต์เคลื่อนที่ผ่านบริเวณที่เกิดการบดบังทางสายตาอย่างรุนแรง หรือสถานะแสงเปลี่ยนไปจนกล้องไม่สามารถติดตามจุดเด่นเดิมได้ (Tracking Lost) ระบบ SLAM ทั่วไปมักจะล้มเหลวและหยุด

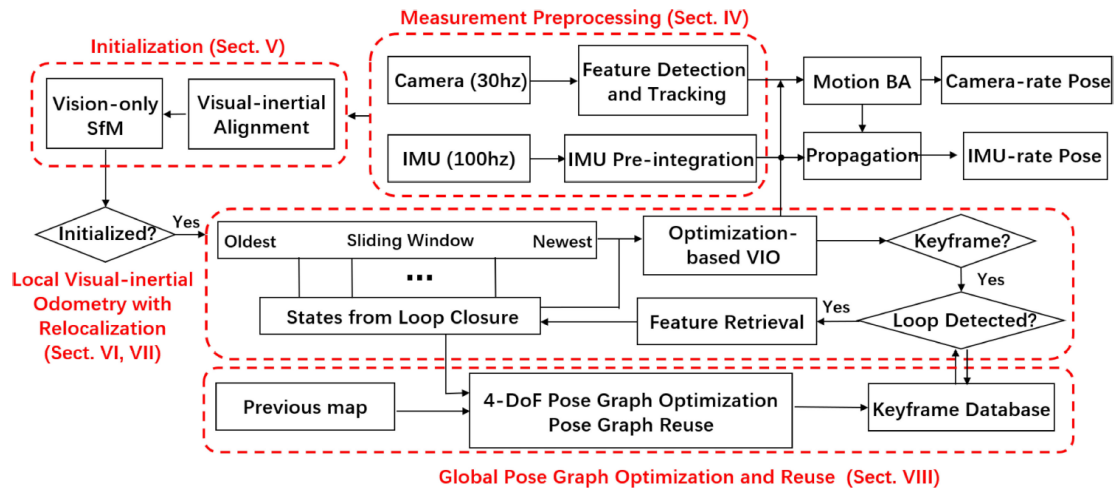
ทำงาน แต่สำหรับ ORB-SLAM3 ระบบจะทำการย้ายแผนที่เดิมที่สูญเสียการติดตามไปไว้ในกลุ่ม Non-active Maps และเริ่มต้นสร้าง Active Map ผืนใหม่ขึ้นมาเพื่อทำหน้าที่ระบุตำแหน่งต่อเนื่องกันที่ จากนั้น เมื่อหุ่นยนต์เคลื่อนที่กลับเข้าสู่พื้นที่เดิมที่เคยสร้างไว้ในอดีต อัลกอริทึมการรู้จำสถานที่ (Place Recognition) ที่ถูกปรับปรุงประสิทธิภาพการดึงกลับ (Improved Recall) จะตรวจพบการซ้อนทับกันของพื้นที่ และสั่งการให้ระบบทำการ ผสานแผนที่ (Map Merging) ระหว่าง Active Map ปัจจุบันและ Non-active Map ในอดีตเข้าด้วยกันโดยอัตโนมัติ พร้อมทั้งปรับแก้ค่าความคลาดเคลื่อนด้วยวิธี Loop Closure กลไกนี้ช่วยลดความเสี่ยงที่หุ่นยนต์จะหลงทางในคลังสินค้าได้อย่างถาวร

4) การแยกส่วนการแทนค่ากล้องเพื่อรองรับเลนส์มุมกว้าง (Abstract Camera Representation):

เนื่องจากในโครงการนี้มีการเลือกใช้กล้องรุ่น AC-IMX390-H190 ที่มีมุมมองรับภาพกว้างพิเศษถึง 186 องศา (Fisheye Lens) ORB-SLAM3 จึงตอบโต้ทางเทคนิคเป็นอย่างมาก เนื่องจากตัวระบบได้รับการออกแบบสถาปัตยกรรมแบบแยกส่วน โดยมีชั้นความหมายสัญลักษณ์การ (Abstract representation) คั่นกลางระหว่างตัวสเก็พเฟเจอร์หลักกับฟังก์ชันการฉายแสงของกล้อง (Camera Projection Model) ทำให้ระบบสามารถเปลี่ยนไปใช้โมเดลกล้องตาปลาแบบ Kannala-Brandt ได้โดยตรง โดยไม่กระทบต่ออัลกอริทึมการคำนวณด้าน SLAM ส่วนหลัง (Back-end) ส่งผลให้หุ่นยนต์สามารถประมวลผลข้อมูลภาพจากกล้องมุมกว้างได้อย่างถูกต้องและไม่มีความบิดเบือนในผลลัพธ์การระบุตำแหน่ง

2.2.3 VINS (Visual-Inertial System)

VINS (Visual-Inertial System เช่น VINS-Mono หรือ VINS-Fusion) เป็นกรอบการทำงานสำหรับการประเมินสถานะและตำแหน่งของหุ่นยนต์ (State Estimator) แบบโอเพนซอร์สที่มีประสิทธิภาพสูง พัฒนาโดยห้องปฏิบัติการหุ่นยนต์ขั้นสูงและการบินอัตโนมัติ (ARA Lab) แห่งมหาวิทยาลัยวิทยาศาสตร์และเทคโนโลยีฮ่องกง (HKUST) แม้สถาปัตยกรรมในรุ่นขยายจะสามารถรองรับการผสานข้อมูลพิกัดดาวเทียม (GPS) ได้ แต่สำหรับการประยุกต์ใช้ในคลังสินค้าในร่มแบบพลวัต (Indoor Dynamic Warehouse) ของโครงการนี้ จะมุ่งเน้นการใช้งาน VINS ในฐานะระบบประเมินตำแหน่งด้วยภาพร่วมกับเซนเซอร์ความเฉื่อยท้องถิ่น (Local Visual-Inertial Odometry: VIO) ที่มีองศาอิสระ 6 มิติ (6-DOF) โดยมีสถาปัตยกรรมเชิงลึกและกลไกการทำงานหลักดังต่อไปนี้



รูปที่ 6 องค์ประกอบหลักในระบบ Monocular VINS

1) กระบวนการประมาณค่าข้อมูลความเฉื่อยล่วงหน้า (IMU Pre-integration):

ในระบบเซนเซอร์ร่วม มาตรวัดความเร่ง (Accelerometer) และไจโรสโคป (Gyroscope) ของ IMU จะส่งข้อมูลด้วยความถี่ที่สูงมาก (เช่น 100-200 Hz) ในขณะที่กล้องส่งข้อมูลภาพด้วยความถี่ที่ต่ำกว่า (เช่น 20-30 Hz) หากระบบ SLAM ทั่วไปทำการคำนวณอินทิเกรตข้อมูล IMU ใหม่ทุกครั้งที่มีการปรับแต่งค่าตำแหน่ง ระบบจะเผชิญกับภาระการประมวลผลที่มหาศาล VINS จึงแก้ปัญหานี้ด้วยการนำเสนอ ทฤษฎีการหาค่าความเร่งล่วงหน้า (IMU Pre-integration) โดยทำการอินทิเกรตข้อมูลความเร่งและความเร็วเชิงมุมระหว่างเฟรมภาพสองเฟรมที่อยู่ติดกันให้อยู่ในกรอบอ้างอิงส่วนท้องถิ่นของตัวหุ่นยนต์ (Local body frame) แทนการใช้กรอบอ้างอิงโลก (Global frame) กลไกนี้ทำให้ค่า Pre-integrated measurements ที่ได้ไม่ขึ้นอยู่กับสถานะพิกัดเริ่มต้น ส่งผลให้ระบบไม่ต้องคำนวณอินทิเกรตใหม่ซ้ำๆ เมื่อตำแหน่งของหุ่นยนต์ถูกปรับแก้ในระบบส่วนหลัง (Back-end optimization) นอกจากนี้ ระบบยังมีการคำนวณการแพร่กระจายความคลาดเคลื่อน (Covariance propagation) และการชดเชยค่าความคลาดเคลื่อนเชิงศูนย์ (Bias correction) ของไจโรสโคปและมาตรวัดความเร่งผ่านการประมาณค่าแบบอันดับหนึ่ง (First-order Taylor expansion) ทำให้การประเมินตำแหน่งมีความแม่นยำทางฟิสิกส์สูง

2) กระบวนการเริ่มต้นระบบแบบผูกแน่น (Robust Estimator Initialization):

ความท้าทายที่สำคัญของระบบ Visual-Inertial SLAM คือการเริ่มต้นระบบ (Initialization) เนื่องจากข้อมูลจากกล้องเลนส์เดี่ยวหรือสเตอริโอไม่มีสเกลระยะทางที่แท้จริงในตอนเริ่มต้น และเซนเซอร์ IMU ก็มีค่า Bias สะสม VINS จึงสถาปนากระบวนการเริ่มต้นระบบแบบผูกแน่น (Tightly-coupled Initialization) ผ่านขั้นตอนย่อยดังนี้:

- **การทำ Structure from Motion (SfM) ส่วนท้องถิ่น:** ระบบจะทำการสกัดจุดเด่นและการจับคู่ภาพ (Feature tracking) ในหน้าต่างเวลาส่วนหนึ่งเพื่อสร้างโครงสร้างสามมิติและตำแหน่งสัมพัทธ์ของกล้องขึ้นมาก่อน

- **การจับคู่แนวแกนและความเฉื่อย (Visual-Inertial Alignment):** ระบบจะนำเส้นทางที่คำนวณได้จาก SfM มาตั้งสมการร่วมกับผลลัพธ์จาก IMU Pre-integration เพื่อหาค่าพารามิเตอร์เริ่มต้น 3 ตัว ได้แก่ สเกลระยะทางที่ถูกต้อง (Scale metric), ทิศทางของแรงโน้มถ่วงของโลกเทียบกับตัวหุ่นยนต์ (Gravity vector) และค่า Bias เริ่มต้นของไจโรสโคป กระบวนการนี้ช่วยให้หุ่นยนต์ AMR สามารถรับรู้ขนาดระยะทางในหน่วยเมตรจริงของคลั่งสินค้าได้ทันทีหลังจากเริ่มเคลื่อนที่

3) การหาค่าพารามิเตอร์ที่เหมาะสมที่สุดผ่านหน้าต่างเลื่อน (Tightly-coupled Sliding Window Optimization):

หัวใจของไปป์ไลน์การคำนวณใน VINS คือการใช้หน้าต่างเลื่อน (Sliding Window) เพื่อจำกัดจำนวนเฟรมภาพและสถานะของหุ่นยนต์ที่ต้องนำมาคำนวณ โดย VINS จะสร้างสมการต้นทุน (Cost function) ในรูปแบบการหาค่าที่เหมาะสมที่สุดแบบไม่เชิงเส้น (Nonlinear least-squares optimization) ผ่านตัวแก้ปัญหาคeres Solver ซึ่งฟังก์ชันต้นทุนจะประกอบด้วยผลรวมของค่าความคลาดเคลื่อน 3 ส่วนผูกแน่นเข้าด้วยกัน ดังสมการ

$$\min_x \{ ||r_p - H_p X||^2 + \sum_{K \in B} ||r_B(z_{b_k b_{k+1}}, X)||_{P_{b_k b_{k+1}}}^2 + \sum_{(l,j) \in C} ||r_C(z_{c_l^j})||_{P_{c_l^j}}^2 \}$$

- **Marginalization Residual (r_p):** ข้อมูลความน่าจะเป็นล่วงหน้า (Prior) ที่ได้มาจากเฟรมภาพในอดีตที่ถูกคัดออกไปจากหน้าต่างเลื่อน
- **IMU Prediction Residual (r_B):** ความคลาดเคลื่อนระหว่างสถานะจริงของหุ่นยนต์กับค่าที่ได้จากการอินทิเกรตล่วงหน้าของ IMU
- **Visual Re-projection Residual (r_C):** ความคลาดเคลื่อนจากการฉายภาพซ้ำของจุดเด่น (Feature points) บนระนาบกล้องสเตอริโอ

4) กลไก Marginalization สำหรับการควบคุมภาวะการคำนวณแบบเรียลไทม์:

เพื่อรักษาทรัพยากรของหน่วยประมวลผลระดับเอดจ์ (NVIDIA Jetson Nano) บนตัวหุ่นยนต์ให้ทำงานแบบเรียลไทม์ได้ตลอดเวลา VINS ได้ออกแบบกลไกการคัดเฟรมภาพออกจากหน้าต่างเลื่อน (Marginalization) อย่างชาญฉลาดโดยแบ่งเป็น 2 เงื่อนไข:

- **Marginalization Old (MARGIN_OLD):** หากเฟรมภาพล่าสุดที่เข้ามาถูกประเมินว่าเป็น "เฟรมหลัก" (Keyframe) ระบบจะทำการคัดเฟรมที่เก่าที่สุดในหน้าต่างเลื่อนออก โดยจะนำข้อมูลเชิงแสงและสถานะความเฉื่อยของเฟรมนั้นมาแปลงเป็นข้อมูลล่วงหน้า (Prior factor) ผ่านระเบียบวิธี Schur complement ทำให้ระบบไม่สูญเสียพารามิเตอร์สำคัญในอดีต
- **Marginalization Second Newest (MARGIN_SECOND_NEW):** หากเฟรมภาพล่าสุดถูกประเมินว่าเป็น "เฟรมปกติที่ไม่ใช่เฟรมหลัก" (Non-keyframe due to lack of parallax) ระบบ

จะทำการลบเฟรมภาพก่อนหน้าล่าสุด (Second newest frame) ที่ทิ้งไปทันที โดยไม่มีการเก็บค่าเป็น Prior กลไกนี้ช่วยคุมขนาดความหนาแน่นของเมทริกซ์และประหยัดการคำนวณได้มหาศาล เมื่อหุ่นยนต์ AMR หยุดนิ่งหรือเคลื่อนที่ช้าในคลังสินค้า

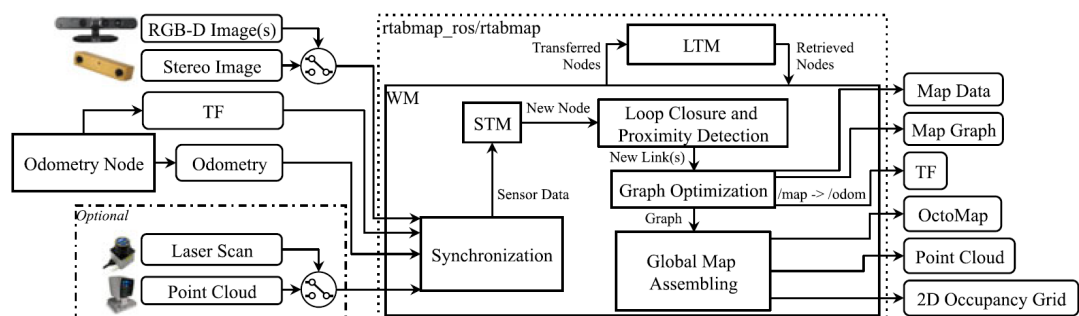
5) การปรับเทียบเวลาแบบออนไลน์ (Online Temporal Calibration):

คุณสมบัติเด่นที่เป็นเอกลักษณ์ของ VINS (อ้างอิงตาม Qin & Shen, 2018) คือการแก้ปัญหา ความไม่ตรงกันของตราประทับเวลา (Temporal misalignment / Time offset) ในความเป็นจริง สัญญาณพัลส์ที่ส่งให้กล้องถ่ายภาพและสัญญาณจากเซนเซอร์ IMU มักเกิดความล่าช้าจากการส่งผ่านข้อมูล (Transmission and triggering delays) ทำให้เวลาของทั้งสองเซนเซอร์ไม่ตรงกัน ซึ่งเป็นตัวการสำคัญที่ทำให้ระบบ VIO ล้มเหลวเมื่อหุ่นยนต์เคลื่อนที่เร็วหรือเลี้ยวโค้ง

VINS จึงนำเสนอโมเดลการคำนวณพารามิเตอร์ความล่าช้าของเวลาเข้าไปเป็นหนึ่งในตัวแปรสถานะที่ต้องปรับปรุ้ค่าร่วมกันในสมการ Optimization ส่วนหลังแบบเรียลไทม์ อัลกอริทึมจะคำนวณการเลื่อนตำแหน่งเฟรมภาพเชิงเวลาเทียบกับความเร็วเชิงมุมของ IMU ส่งผลให้ไปป์ไลน์สามารถชดเชยค่าความหน่วงของสตรีมภาพกล้อง AC-IMX390-H190 บนบอร์ดประมวลผลได้อย่างแม่นยำ ช่วยรักษาเสถียรภาพของการระบุตำแหน่ง (Tracking consistency) แม้หุ่นยนต์ต้องเผชิญกับการสะท้อนบนพื้นคลังสินค้าที่มีความลาดชัน 5 องศา

2.2.4 ระบบจัดการหน่วยความจำและการสร้างแผนที่ระยะยาวด้วย RTAB-Map (RTAB Map Back-End & Memory Management)

อ้างอิงจากงานวิจัยของ Labbé & Michaud (2018) RTAB-Map เป็นไลบรารีระบบ SLAM แบบโอเพนซอร์สที่ถูกออกแบบมาเพื่อแก้ปัญหาคอมพิวเตอร์และหน่วยความจำระเบิด (Computational & Memory Explosion) เมื่อหุ่นยนต์ต้องทำงานในพื้นที่ขนาดใหญ่หรือทำงานต่อเนื่องเป็นเวลานาน (Lifelong Mapping) โดย RTAB-Map รับข้อมูล Odometry และ Point Cloud จาก VINS มาทำ Graph SLAM ส่วนหลังผ่านกลไกหลัก 3 ประการ:



รูปที่ 7 องค์ประกอบหลักในระบบ RTAB-MAP

1) สถาปัตยกรรมแบ่งชั้นหน่วยความจำตามแบบโมเดลสมองมนุษย์ (Memory Partitioning Model):

RTAB-Map ไม่ได้เก็บและคำนวณโหนดแผนที่ทั้งหมดไว้ในหน่วยความจำชั่วคราว (RAM) แต่จัดระเบียบข้อมูลออกเป็น 3 ระดับ:

- **Short-Term Memory (STM):** ทำหน้าที่รับข้อมูลภาพและตำแหน่ง Odometry ล่าสุดจาก VINS เพื่อสกัดจุดเด่น (Feature Extraction) และสร้างเป็นโหนด (Node) ใหม่ โหนดใน STM จะยังไม่ถูกนำไปใช้คำนวณหาการวนกลับมาที่เดิม (Loop Closure)
- **Working Memory (WM) :** เป็นพื้นที่หลักใน RAM ที่อัลกอริทึมใช้ประมวลผลเพื่อตรวจจับ Loop Closure และทำ Graph Optimization โดยระบบจะทำการ จำกัดขนาดของ WM ไว้คงที่ผ่านเกณฑ์ขอบเขตเวลา (Time Threshold) เพื่อรับประกันว่าเวลาในการคำนวณจะไม่เกินข้อจำกัดเรียลไทม์ (เช่น < 30 มิลลิวินาที)
- **Long-Term Memory (LTM) :** คือฐานข้อมูล (Database) ที่ถูกบันทึกลงในหน่วยความจำถาวร (SSD/Hard Disk) ข้อมูลในส่วนนี้จะไม่ถูกนำมาคำนวณในสมการปรับแต่งกราฟปกติ เพื่อสงวนทรัพยากร RAM และ CPU

2) อัลกอริทึมการถ่ายโอนและถ่วงน้ำหนักข้อมูล (Memory Transfer & Weighting Heuristics):

เพื่อให้ความซับซ้อนของเวลาในการประมวลผลถูกล็อกไว้เป็นค่าคงที่ ($O(1)$ Time Complexity) ไม่ว่าแผนที่คลัสเตอร์จะมีขนาดใหญ่เพียงใด RTAB-Map ใช้กลไกย้ายข้อมูลไปมาระหว่าง WM และ LTM ดังนี้:

- **การเพิ่มน้ำหนักโหนด (Node Weighting) :** เมื่อหุ่นยนต์เคลื่อนที่ผ่านเส้นทางเดิมและเกิด Loop Closure โหนดบริเวณนั้นจะได้รับคะแนนน้ำหนัก (Weight) เพิ่มขึ้น โหนดที่มีการสัญจรผ่านบ่อยครั้ง (เช่น ทางเดินหลักในคลังสินค้า) จะมีน้ำหนักสูง
- **การย้ายข้อมูลสู่ความจำระยะยาว (WM to LTM Transfer) :** เมื่อเวลาประมวลผลใน WM เริ่มเข้าใกล้ขีดจำกัดที่ตั้งไว้ อัลกอริทึมจะบังคับย้ายโหนดที่ "เก่าที่สุดและมีน้ำหนักน้อยที่สุด" ออกจาก RAM ไปจัดเก็บไว้ใน LTM (SSD) ชั่วคราว
- **การฟื้นความทรงจำ (LTM to WM Retrieval):** หากหุ่นยนต์สัญจรกลับเข้าสู่พื้นที่เก่า และระบบสกัดจุดเด่นพบความเชื่อมโยงกับโหนดในอดีต อัลกอริทึมจะทำการดึง (Retrieve) โหนดข้างเคียงที่เกี่ยวข้องจาก LTM กลับเข้ามาใน WM โดยอัตโนมัติ เพื่อเชื่อมต่อโครงสร้างแผนที่และปรับแก้ค่าความคลาดเคลื่อนสะสม (Drift)

3) การปรับแต่งโครงข่ายกราฟภายใต้ข้อจำกัดหน่วยความจำ (Memory-Constrained Graph Optimization) :

ต่างจากระบบ SLAM ทั่วไปที่ต้องคำนวณปรับแต่งสมการกราฟทั้งพื้นแผนที่ (Global Graph Optimization) ซึ่งทำให้เครื่องค้างเมื่อวิ่งเป็นเวลานาน RTAB-Map จะประมวลผลสมการปรับแต่งกราฟ (ผ่านตัวแก้ปัญหา g2o หรือ GTSAM) เฉพาะ โหนดที่หลงเหลืออยู่ใน Working Memory (WM)

เท่านั้น ส่งผลให้หน่วยประมวลผลระดับเอดจ์ NRU-51V สามารถรันระบบนำทางและอัปเดตแผนที่ คลังสินค้าได้อย่างลื่นไหลตลอดอายุการใช้งาน โดยที่ความเร็วไม่ตกลง

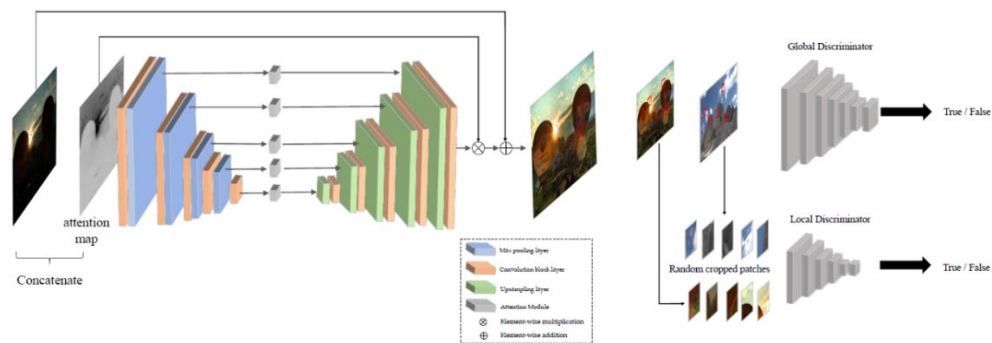
2.3 การปรับปรุงคุณภาพภาพในสถานะแสงน้อย (Image Enhancement)

ข้อจำกัดที่สำคัญที่สุดประการหนึ่งของระบบการระบุตำแหน่งและการสร้างแผนที่ด้วยภาพ (Visual SLAM) คือความอ่อนไหวต่อสภาพแวดล้อมที่มีการเปลี่ยนแปลงของความเข้มแสง (Illumination changes) หรือสถานะแสงสว่างไม่เพียงพอ (Low-light conditions) ภายในคลังสินค้า อัลกอริทึมการสกัดจุดเด่น (Feature Extraction) เช่น ORB หรือ FAST มักจะประสบความล้มเหลวในการค้นหาและจับคู่จุดเด่นเมื่อภาพมีความมืด ความเปรียบต่างต่ำ (Low contrast) หรือมีจุดรบกวน (Noise) สูง ส่งผลให้ระบบสูญเสียการติดตามตำแหน่ง (Tracking lost) และทำให้หุ่นยนต์หลงทางได้ ดังนั้น โครงการงานนี้จึงบูรณาการระบบปรับปรุงคุณภาพภาพ (Image Enhancement) เข้ามาเป็นส่วนประมวลผลล่วงหน้า (Preprocessing) ก่อนส่งข้อมูลภาพเข้าสู่ไปป์ไลน์หลัก

2.3.1 สถาปัตยกรรมโครงข่ายประสาทเทียม EnlightenGAN

เพื่อหลีกเลี่ยงข้อจำกัดของการปรับแต่งภาพแบบดั้งเดิม โครงการงานนี้จึงเลือกประยุกต์ใช้โมเดลโครงข่ายประสาทเทียม EnlightenGAN อ้างอิงจากงานวิจัยของ Jiang et al. (2021) ซึ่งเป็นสถาปัตยกรรมแบบ Generative Adversarial Networks (GAN) ที่ออกแบบมาเพื่อเพิ่มความสว่างและปรับปรุงคุณภาพภาพถ่ายในสถานะแสงน้อยโดยเฉพาะ โดยมีกลไกทางเทคนิคที่สำคัญดังนี้:

- **การเรียนรู้แบบไม่ต้องอาศัยข้อมูลคู่ขนาน (Unpaired Supervision):** โมเดลไม่จำเป็นต้องใช้ชุดข้อมูลฝึกสอนแบบคู่ขนาน (Paired data) ซึ่งเป็นไปได้ยากในการจัดเก็บในคลังสินค้าจริง แต่ใช้ข้อมูลจากตัวภาพอินพุตเองมาเป็นตัวกำกับ ทำให้สามารถนำภาพคลังสินค้าในสถานะแสงใดๆ มาใช้ฝึกสอนโมเดลได้โดยตรง
- **การใช้ตัวแยกแยะแบบคู่ (Global-Local Discriminators):** ใช้ตัวแยกแยะ 2 ระดับในการประเมินความสมจริงของภาพรวมและพื้นที่ย่อย (Patches) เพื่อกระจายแสงและลดจุดรบกวน (Noise) ในบริเวณที่เคยมีดสนิทได้อย่างเป็นธรรมชาติ
- **กลไกความสนใจและฟังก์ชันการสูญเสียแบบกำกับตนเอง (Attention Mechanism and Self-Regularized Loss):** ระบบจะคำนวณแผนผังความสนใจจากช่องว่างของความสว่าง ช่วยให้โมเดลมุ่งเน้นการเพิ่มแสงเฉพาะใน "บริเวณที่มีด" และรักษาสภาพความสว่างในบริเวณที่ปกติเอาไว้ เพื่อป้องกันการเกิดสถานะแสงสว่างจ้าเกินไป (Overexposure)



รูปที่ 8 สถาปัตยกรรมของระบบ EnlightenGAN

2.4 การตรวจจับวัตถุแบบเรียลไทม์ (Real-Time Object Detection)

2.4.1 บทบาทของการตรวจจับวัตถุในระบบความหมายเชิงลึก (Semantic Understanding)

ในระบบการนำทางด้วยภาพแบบดั้งเดิม (Geometric Visual SLAM) อัลกอริทึมจะสกัดจุดเด่นทางเรขาคณิตโดยไม่รับรู้ถึงประเภทของวัตถุ อย่างไรก็ตาม สำหรับคลังสินค้าที่มีการเปลี่ยนแปลงตลอดเวลา (Dynamic Warehouse) การรับรู้เพียงแค่ข้อมูลเรขาคณิตนั้นไม่เพียงพอ ระบบจำเป็นต้องมีความเข้าใจเชิงความหมาย (Semantic Understanding) เพื่อแยกแยะว่าวัตถุใดคือโครงสร้างถาวร และวัตถุใดคือสิ่งกีดขวางพลวัตที่สามารถเคลื่อนย้ายได้ (เช่น พallet พาเลท หรือบุคคล) ดังนั้น โครงงานนี้จึงบูรณาการระบบการตรวจจับวัตถุ (Object Detection) เข้ามาเป็นส่วนประมวลผลส่วนหน้า (Frontend) เพื่อสร้างกรอบล้อมรอบ (Bounding Box) และส่งพิกัดของวัตถุพลวัตไปให้ระบบ SLAM ทำการสกัดกัน (Masking) จุดเด่นที่ไม่เสถียรออกไป

2.4.2 สถาปัตยกรรมโครงข่ายประสาทเทียมแบบ YOLO (You Only Look Once)

เพื่อให้ระบบทำงานตอบสนองได้ทันทั่วทั้งที่ (Real-time constraints) ควบคู่ไปกับการรันระบบ SLAM โครงงานนี้จึงมุ่งเน้นศึกษาโมเดลในตระกูล YOLO ซึ่งเป็นสถาปัตยกรรมการตรวจจับวัตถุแบบขั้นตอนเดียว (One-stage Detector) ที่มีความโดดเด่นด้านความเร็วในการประมวลผล ต่างจากโมเดลแบบสองขั้นตอน (เช่น Faster R-CNN) ที่ต้องสร้างขอบเขตที่น่าสนใจ (Region Proposals) ก่อนแล้วจึงจำแนกประเภท YOLO จะทำการทำนายพิกัดกรอบล้อมรอบ (Bounding box regression) และความน่าจะเป็นของคลาส (Class probabilities) จากภาพรวมทั้งหมดภายในรอบการส่งผ่านข้อมูลเพียงครั้งเดียว (Single forward pass) ทำให้ YOLO (เช่น YOLOv26) เหมาะสมอย่างยิ่งสำหรับการติดตั้งบนหน่วยประมวลผลระดับเอดจ์ (Edge AI) อย่างบอร์ด NRU-51V ที่มีทรัพยากรจำกัด

ตารางที่ 1 แสดงการเปรียบเทียบสถาปัตยกรรมโมเดลตรวจจับวัตถุ

สถาปัตยกรรมโมเดล	หมวดหมู่	จุดแข็งหลัก	ข้อจำกัด	กรณีการใช้งานที่เหมาะสม
Faster R-CNN	โมเดลตรวจจับแบบสองขั้นตอน (Two-Stage Detector)	โดดเด่นด้านความแม่นยำในการแยกประเภท (Classification) และการระบุตำแหน่ง (Localization) ทำค่า mAP และ IoU ได้สูง มีอัตราข้อผิดพลาดโดยรวมต่ำและมีความแม่นยำเชิงพื้นที่สูง	กระบวนการทำงานแบบหลายขั้นตอนทำให้ความเร็วในการประมวลผล (Inference speed) ช้าลง	งานที่ต้องการความแม่นยำสูง และไม่จำเป็นต้องประมวลผลแบบเรียลไทม์ (Real-time)
YOLO Series	โมเดลตรวจจับแบบขั้นตอนเดียว (Single-Stage Detector)	ปรับวิธีตรวจจับให้เป็นปัญหา Regression แบบขั้นตอนเดียว เน้นปรับแต่งเพื่อความเร็ว รักษาสมดุลระหว่างความเร็วและความแม่นยำได้ดีเยี่ยม ให้ค่า Precision และ Recall สูง และมีอัตราตรวจจับผิดพลาด (FPPI) ต่ำ	แม้ปัจจุบันจะปรับแต่งมาอย่างดี แต่เวอร์ชันแรกๆ มักมีปัญหาในการตรวจจับวัตถุขนาดเล็ก หรืออยู่กระจุกตัวหนาแน่น เมื่อเทียบกับแบบสองขั้นตอน	งานแบบเรียลไทม์ที่ต้องการความเร็วในการประมวลผลสูงและมีความหน่วง (Latency) ต่ำ
SSD (Single Shot Detectors)	โมเดลตรวจจับแบบขั้นตอนเดียว (Single-Stage Detector)	ตัดขั้นตอนการเสนอพื้นที่ (Region proposal) ออกไป และรวมการคำนวณเพื่อตรวจจับวัตถุหลายขนาด (Multi-scale) ให้จบภายในเครือข่ายเดียว	บางครั้งความเร็วอาจเป็นรอง YOLO เล็กน้อย แต่ก็ยังถือว่ามีประสิทธิภาพสูงมาก	แอปพลิเคชันที่ต้องการความสมดุลที่เชื่อถือได้ระหว่างประสิทธิภาพการประมวลผลและความแม่นยำ
MobileNet / EfficientNet	โมเดลขนาดเล็กเบา (Lightweight Model)	ใช้เทคนิคอย่าง Depthwise Separable Convolutions เพื่อลดพารามิเตอร์และการคำนวณลงอย่างมหาศาล โดยไม่ลดทอนประสิทธิภาพการเรียนรู้ของโมเดล	ความแม่นยำโดยรวมอาจต่ำกว่าโมเดลขนาดใหญ่ อย่าง Faster R-CNN หรือ YOLO เวอร์ชันใหญ่	อุปกรณ์ Edge ที่มีหน่วยความจำและพลังประมวลผลจำกัด เช่น สมาร์ทโฟน อุปกรณ์ IoT และ โดรน อัดโนมัติ

2.4.3 การประเมินประสิทธิภาพและข้อจำกัดบนหน่วยประมวลผลระดับเอดจ์

ในการนำโมเดล YOLO มาประยุกต์ใช้ในไปป์ไลน์ Visual SLAM สิ่งสำคัญคือการรักษาสมดุล (Trade-off) ระหว่างความแม่นยำและระยะเวลาในการอนุมาน (Inference time / Latency) เนื่องจากหากใช้โมเดลที่มีขนาดใหญ่เกินไป อาจทำให้เกิดปัญหาคอขวดและส่งผลกระทบต่อระบบ SLAM ทำงานล่าช้ากว่าอัตราเฟรมเรตของกล้อง (Camera Frame Rate) ได้ ดังนั้น การประเมินประสิทธิภาพของโมเดล YOLO บนชุดข้อมูลจำลองคลังสินค้าผ่านแพลตฟอร์มอย่าง Roboflow จะพิจารณาจากตัวชี้วัดหลัก (Evaluation Metrics) ดังนี้

- **Intersection over Union (IoU) :** อัตราส่วนพื้นที่ที่ทับซ้อนระหว่างกรอบล้อมรอบที่โมเดลทำนายได้กับกรอบวัตถุจริง เพื่อวัดความแม่นยำของตำแหน่งวัตถุ

- **Precision** : สัดส่วนของการทำนายวัตถุพลวัตได้ถูกต้อง จากจำนวนการทำนายว่าเป็นวัตถุพลวัตทั้งหมด (วัดความแม่นยำของการทายผล)
- **Recall** : สัดส่วนของการทำนายวัตถุพลวัตได้ถูกต้อง จากจำนวนวัตถุพลวัตที่มีอยู่จริงทั้งหมด ในภาพ (วัดความครอบคลุมในการค้นหา)
- **F1 Score**: ค่าเฉลี่ยฮาร์โมนิก (Harmonic Mean) ระหว่าง Precision และ Recall เพื่อดูความสมดุลของโมเดล
- **Average Precision (AP) / Mean Average Precision (mAP)**: ค่าเฉลี่ยความแม่นยำรวมที่สะท้อนประสิทธิภาพเชิงรวมของโมเดลในการตรวจจับวัตถุทุกประเภทภายใต้เงื่อนไขความมั่นใจ (Confidence threshold) ที่แตกต่างกัน

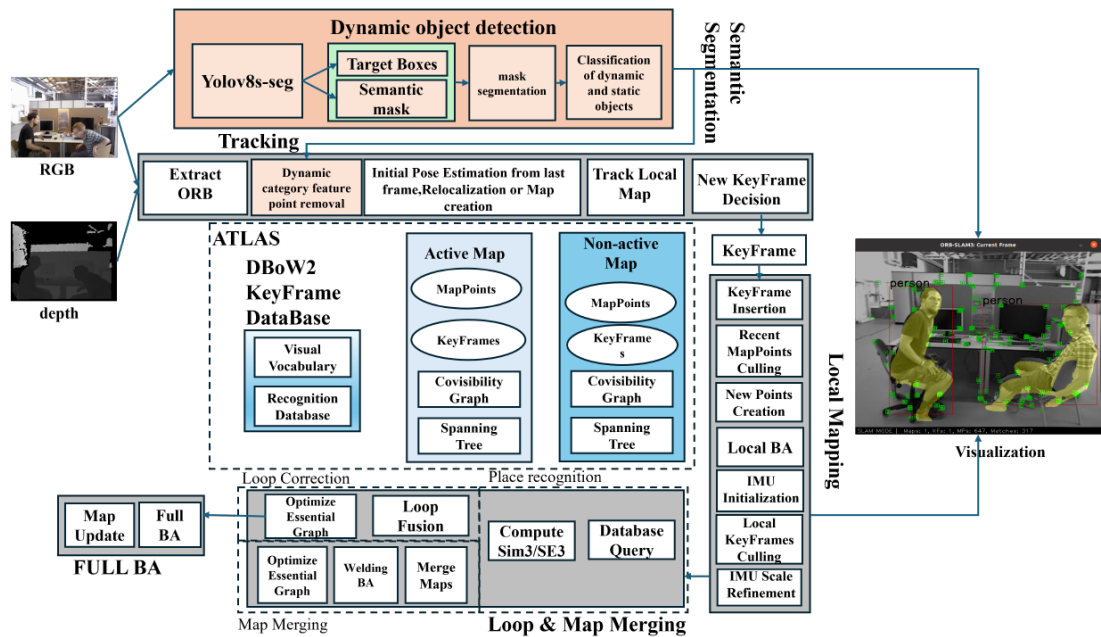
การเลือกโมเดลที่ให้ค่า mAP สูงสุดโดยที่ยังคงรักษาค่า Latency ให้อยู่ในกรอบเวลาที่ปลอดภัย (เช่น < 15-20 ms) ถือเป็นกุญแจสำคัญที่จะทำให้การผสานการทำงานระหว่าง YOLO และ SLAM เป็นไปอย่างสมบูรณ์

2.5 RY-SLAM: A robust YOLO-based real-time visual SLAM solution for dynamic environments

2.5.1 แนวคิดและสถาปัตยกรรมหลักของ RY-SLAM

อ้างอิงจากงานวิจัยของ Chen et al. (2025) ได้นำเสนอกรอบการทำงาน RY-SLAM (Robust YOLO SLAM) ซึ่งเป็นระบบ Visual SLAM ขั้นสูงที่ถูกออกแบบมาเพื่อแก้ปัญหาค่าความล้มเหลวของระบบนำทางเมื่อต้องทำงานในสภาพแวดล้อมที่มีการเคลื่อนไหว (Dynamic Environments) โดย RY-SLAM ได้ทำการบูรณาการกรอบการทำงาน ORB-SLAM3 เข้ากับโครงข่ายประสาทเทียมตรวจจับและแบ่งส่วนภาพเชิงความหมายตระกูล YOLO (เช่น YOLOv8s-seg)

สถาปัตยกรรมของ RY-SLAM ถูกออกแบบให้อาศัยการประมวลผลแบบขนานหลายเธรด (Multi-threading) ร่วมกับการเร่งความเร็วด้วยหน่วยประมวลผลกราฟิก (GPU Acceleration) โดยระบบจะทำการแยกเธรดประมวลผลส่วนการรับรู้เชิงความหมาย (Semantic Segmentation Thread) ออกจากเธรดการติดตามตำแหน่งหลัก (Tracking Thread) ของ SLAM ส่งผลให้ระบบสามารถวิเคราะห์พิกัดวัตถุพลวัตและประเมินตำแหน่งของกล้องไปพร้อมๆ กันได้โดยไม่สูญเสียอัตราการประมวลผลแบบเรียลไทม์

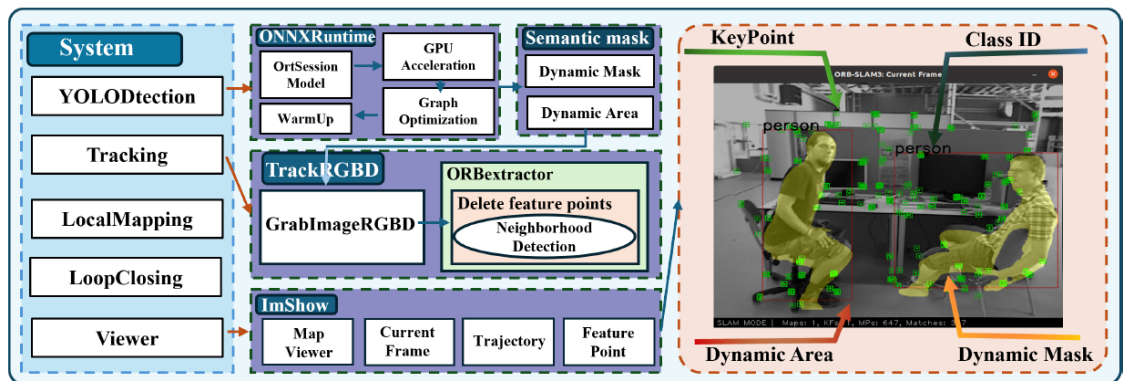


รูปที่ 9 สถาปัตยกรรมของระบบ RY-SLAM

2.5.2 กลไกการคัดกรองจุดเด่นพลวัต (Dynamic Feature Masking)

หัวใจสำคัญของ RY-SLAM ในการรักษาความแม่นยำของการระบุตำแหน่ง คือกระบวนการ Dynamic Feature Masking ซึ่งมีขั้นตอนทางเทคนิคดังนี้:

- **การสร้างแผนผังความหมาย (Semantic Mask Generation):** โมเดล YOLO จะทำการวิเคราะห์เฟรมภาพอินพุตเพื่อตรวจจับวัตถุพลวัต (เช่น บุคคล หรือยานพาหนะ) และสร้างแมสก์ล้อมรอบพื้นที่ของวัตถุดังกล่าว
- **การคัดกรองจุดเด่นที่ไม่เสถียร (Outlier Elimination):** ในขั้นตอนการสกัดจุดเด่น (Feature Extraction) ของ ORB-SLAM3 ระบบจะทำการตรวจสอบตำแหน่งของจุดเด่น (ORB Features) หากจุดเด่นใดตกอยู่ในพื้นที่แมสก์ของวัตถุดพลวัต ระบบจะทำการลบจุดเด่นนั้นทิ้งไปทันที
- **การประเมินตำแหน่งจากฉากหลังคงที่ (Static Pose Estimation) :** อัลกอริทึมจะนำเฉพาะจุดเด่นที่สกัดได้จากโครงสร้างถาวรของสภาพแวดล้อม (Static Background) ไปป้อนให้กับกระบวนการคำนวณปรับแต่งโครงข่ายกราฟ (Bundle Adjustment) ทำให้ค่าความคลาดเคลื่อนสะสม (Drift) และปัญหาการสูญเสียการติดตาม (Tracking Lost) ลดลงอย่างมีนัยสำคัญ เมื่อเทียบกับระบบ ORB-SLAM3 แบบมาตรฐาน



รูปที่ 10 รายละเอียดของเฟรมเวิร์ก Semantic Segmentation ใน RY-SLAM และกระบวนการทำงานแบบ Multi-Threaded

2.5.2 กลไกการคัดกรองจุดเด่นพลวัต (Dynamic Feature Masking)

งานวิจัย RY-SLAM ถือเป็นหลักฐานทางวิชาการสำคัญที่ยืนยันถึงประสิทธิภาพและความสมเหตุสมผลของการผสานโมเดลตระกูล YOLO เข้ากับ ORB-SLAM3 สำหรับโครงการนี้ คณะผู้จัดทำได้นำแนวคิดและสถาปัตยกรรมพื้นฐานของ RY-SLAM มาปรับใช้และพัฒนาต่อยอดใน ไปป์ไลน์รูปแบบที่ 1 (YOLO26 + ORB-SLAM3) โดยโครงการนี้ได้ยกระดับประสิทธิภาพของระบบด้วยการเลือกใช้โมเดลรุ่นใหม่อย่าง YOLO26 เพื่อทำหน้าที่คัดกรองสิ่งกีดขวางและพัสดุจำลองในคลังสินค้าแบบเรียลไทม์ ร่วมกับสถาปัตยกรรม Atlas Multi-map ของ ORB-SLAM3 บนหน่วยประมวลผลระดับเอดจ์ NRU-51V เพื่อสร้างระบบนำทางที่มีทั้งความทนทานต่อวัตถุพลวัตระยะประชิด (Close-Proximity Dynamic Objects) และมีความทนทานในการสร้างแผนที่ระยะยาว (Lifelong Mapping) ตามวัตถุประสงค์ที่ตั้งไว้

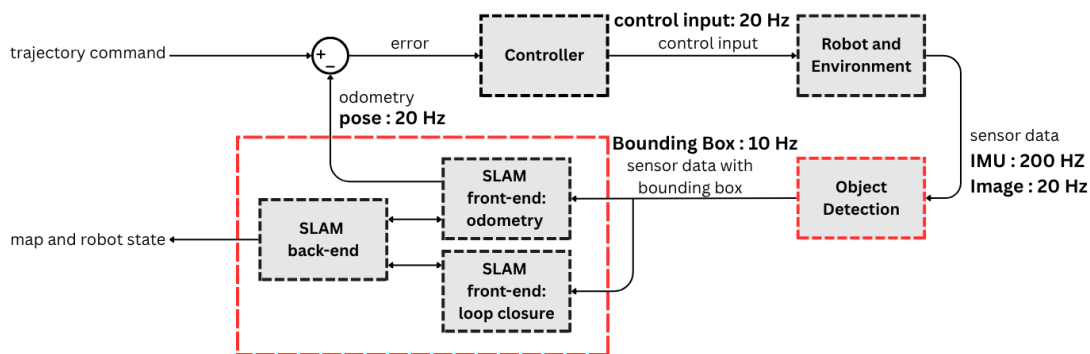
บทที่ 3 ระเบียบวิธีวิจัย

เพื่อให้การพัฒนาระบบสามารถแก้ไขปัญหาดังกล่าวและนำไปประยุกต์ใช้งานได้จริงบนหน่วยประมวลผลระดับเอดจ์ (Edge AI Computing) โครงการนี้จึงได้นำเสนอการบูรณาการโครงข่ายประสาทเทียมเพื่อการตรวจจับวัตถุ (YOLO26) เข้ากับกระบวนการทำงานหลักของ Visual SLAM ซึ่งเรียกว่าระบบ Semantic Visual SLAM โดยในบทนี้จะอธิบายครอบคลุมถึงภาพรวมของสถาปัตยกรรมระบบ การเปรียบเทียบไปป์ไลน์การนำทาง 2 รูปแบบ รายละเอียดฮาร์ดแวร์และเซนเซอร์ที่ใช้ ตลอดจนการออกแบบการทดลองภายใต้สภาพแวดล้อมจำลอง (Experimental Design) และการกำหนดตัวชี้วัดประสิทธิภาพ (Evaluation Metrics) เพื่อประเมินความสมรรถนะของระบบอย่างเป็นรูปธรรม โดยมีรายละเอียดการดำเนินงานในแต่ละส่วนดังต่อไปนี้

3.1 ภาพรวมการออกแบบระบบ (Overall System Architecture)

1) สถาปัตยกรรมระบบนำทางด้วยภาพเชิงความหมาย (Semantic Visual SLAM Architecture)

สถาปัตยกรรมระบบถูกจัดโครงสร้างออกเป็น 3 ส่วนหลัก ได้แก่ ส่วนรับและประมวลผลข้อมูลเซนเซอร์ส่วนหน้า (Frontend & Sensor Processing), ส่วนประมวลผลตำแหน่งและสร้างแผนที่ส่วนหลัง (Backend SLAM & Mapping)



รูปที่ 11 แสดงสถาปัตยกรรมของโครงการโดยสังเขป

- การประมวลผลส่วนหน้า (Frontend Processing Pipeline)

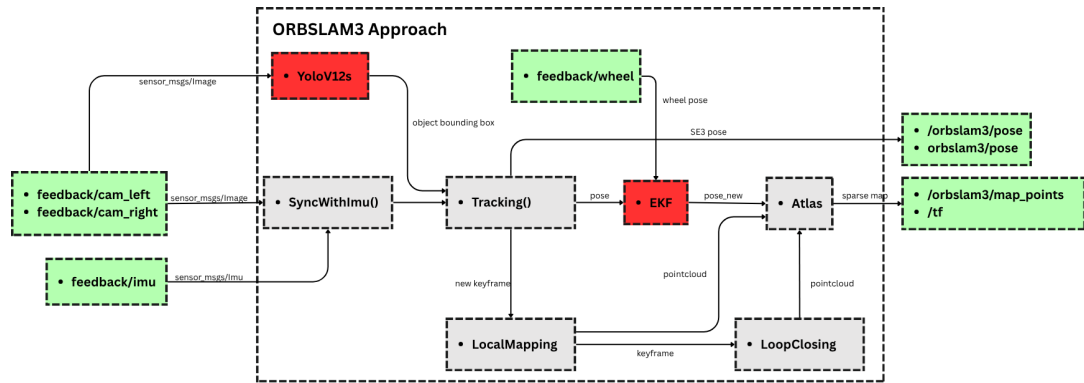
- **Sensor Data Ingestion:** รับข้อมูลสัญญาณภาพดิบจากกล้องสเตอริโอเลนส์มุมกว้าง (Sony IMX390, FOV 186 องศา) ร่วมกับข้อมูลอัตราเร่งและความเร็วเชิงมุมจากหน่วยวัดความเฉื่อย (IMU) ผ่านโหนด ROS2
- **Dynamic Object Detection Module :** ข้อมูลภาพจะถูกส่งไปยังโมเดลปัญญาประดิษฐ์ YOLO26 เพื่อทำการตรวจจับวัตถุพลวัตและสิ่งกีดขวางที่สามารถ

เปลี่ยนแปลงตำแหน่งได้ในคลังสินค้า (เช่น พัสดุ, พาเลท, หรือบุคคล) แบบเรียลไทม์ โดยโมเดลจะส่งออกพิกัดกรอบล้อมรอบวัตถุ (Bounding Boxes)

- **Dynamic Feature Masking Protocol** : ระบบทำการเปรียบเทียบพิกัด Bounding Box ร่วมกับจุดเด่นทางเรขาคณิต (Keypoints/Features) ที่สกัดได้จากอัลกอริทึมสกัดจุดเด่น (เช่น ORB หรือ FAST) หากจุดเด่นใดตกอยู่ในขอบเขตของวัตถุพลวัต ระบบจะทำการสกัดกั้น (Masking) และตัดจุดเด่นนั้นออกจากระบบ เพื่อให้เหลือเฉพาะจุดเด่นของโครงสร้างคลังสินค้าแบบถาวร (Static Background)
- **การประมวลผลส่วนหน้า (Frontend Processing Pipeline)**
 - จุดเด่นที่ผ่านการคัดกรองแล้วจะถูกส่งต่อเข้าสู่กระบวนการคำนวณ Odometry และการทำ Graph Optimization เพื่อประเมินพิกัดสถานะของหุ่นยนต์ (Robot State Estimation) แบบ 6-DOF ควบคู่ไปกับการอัปเดตแผนที่โครงสร้างถาวรในรูปแบบ 3D Point Cloud และการสร้างแผนที่ต้นทุน 2D (Grid Costmap) สำหรับระบบการนำทาง

2) การเปรียบเทียบไปป์ไลน์ระบบนำทาง 2 รูปแบบ (Proposed Navigation Pipelines Comparison)

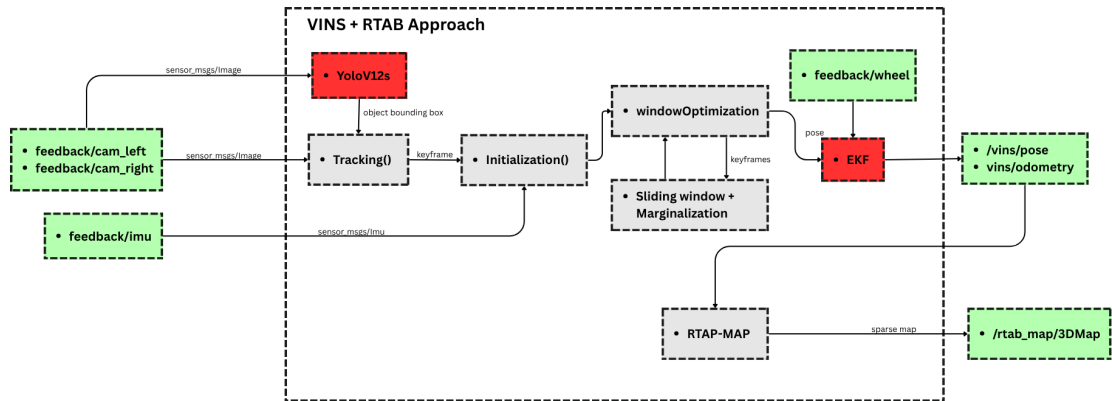
- **รูปแบบที่ 1: ไปป์ไลน์ YOLO26 + ORB-SLAM3 (Atlas Multi-map System)**
 - **แนวคิดสถาปัตยกรรม** : เป็นสถาปัตยกรรมแบบรวมศูนย์ (Tightly-coupled Visual-Inertial SLAM) ที่ทำหน้าที่ทั้งการประเมิน Odometry และสร้างแผนที่เบื้องหลังผ่านโครงข่ายกราฟเดียวกัน
 - **กลไกการจัดการ Lifelong Mapping** : อาศัยระบบ Atlas Multi-map Framework ของ ORB-SLAM3 เมื่อหุ่นยนต์สูญเสียการติดตามตำแหน่ง (Tracking Lost) เนื่องจากการเปลี่ยนแปลงของสิ่งแวดล้อม ระบบจะสลับไปสร้างแผนที่ย่อยใหม่ (Active Map) และเก็บแผนที่เดิมไว้ (Non-active Map) เมื่อหุ่นยนต์กลับเข้าสู่พื้นที่เดิม ระบบจะทำการผสานแผนที่ (Map Merging) และเชื่อมโยงโหนดด้วย Loop Closure ข้ามแผนที่
 - **จุดเด่น** : มีความแม่นยำสูงในการผสานข้อมูลภาพและ IMU แบบ Tightly-coupled เหมาะสำหรับพื้นที่ที่มีคุณลักษณะทางเรขาคณิตชัดเจน



รูปที่ 12 แสดงสถาปัตยกรรมของไปป์ไลน์ YOLO26 + ORB-SLAM3 (Atlas Multi-map System)

- รูปแบบที่ 2 : ไปป์ไลน์ YOLO26 + VINS + RTAB-Map (Decoupled VIO & Memory Management System)

- แนวคิดสถาปัตยกรรม : เป็นสถาปัตยกรรมแบบแยกส่วนภาระงาน (Decoupled Architecture) โดยแบ่งหน้าที่ชัดเจนระหว่างการประเมินตำแหน่งความถี่สูงส่วนหน้า และการจัดการหน่วยความจำระยะยาวส่วนหลัง
- ส่วนประเมินการเคลื่อนที่ (Front-end Odometry) : ใช้กรอบการทำงาน VINS (Visual-Inertial System) รับข้อมูล Static Features จาก YOLO26 มาคำนวณ Visual-Inertial Odometry (VIO) ผ่านกระบวนการ Sliding Window Optimization ซึ่งให้ค่า Odometry ที่มีความหน่วงต่ำมาก (Low Latency) และเสถียรเมื่อหุ่นยนต์สะดุดหรือวิ่งบนพื้นเอียง 5 องศา
- กลไกการจัดการ Lifelong Mapping : รับข้อมูล Odometry จาก VINS เข้าสู่ RTAB-Map ซึ่งใช้กลไกการแบ่งหน่วยความจำแบบ Working Memory (WM) และ Long-Term Memory (LTM) โดยระบบจะคุมขนาดของ WM ให้คงที่เพื่อจำกัดเวลาคำนวณให้อยู่ในเกณฑ์เรียลไทม์ (O(1) Complexity) และถ่ายโอนโหนดเก่าลง SSD (LTM)
- จุดเด่น : มีความเสถียรสูงด้านการจัดการทรัพยากรฮาร์ดแวร์ ป้องกันปัญหา RAM เต็มบนบอร์ดประมวลผลขนาดเล็กเมื่อทำงานต่อเนื่องเป็นเวลานาน



รูปที่ 13 ไปป์ไลน์ YOLO26 + VINS + RTAB-Map (Decoupled VIO & Memory Management System)

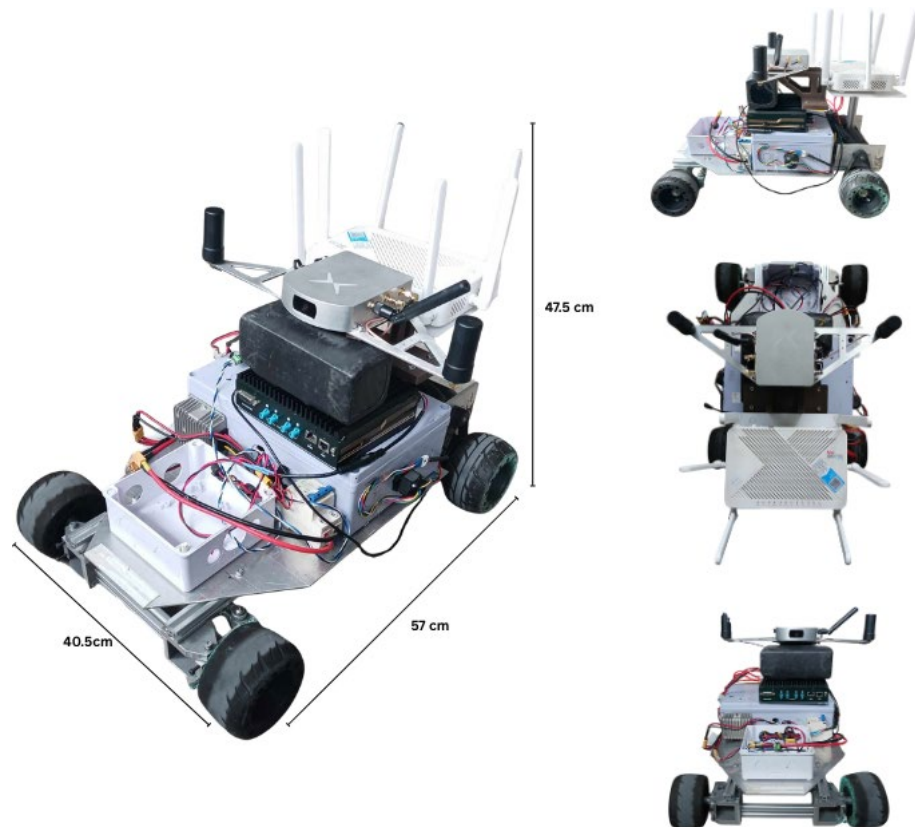
ตารางที่ 2 แสดงการเปรียบเทียบสถาปัตยกรรมไปป์ไลน์ทั้ง 2 รูปแบบ

ประเด็นการเปรียบเทียบ	ไปป์ไลน์ที่ 1: YOLO26 + ORB-SLAM3	ไปป์ไลน์ที่ 2: YOLO26 + VINS + RTAB-Map
โครงสร้างระบบ	Tightly-coupled (รวมศูนย์)	Decoupled (แยกส่วน Odometry และ Backend)
ส่วนประเมิน Odometry	ORB-SLAM3 Tracking Thread	VINS-Fusion / VINS Local VIO
กลไกการทำ Lifelong Mapping	Atlas Multi-map System & Map Merging	RTAB-Map Memory Management (WM / LTM)
ความซับซ้อนเวลาคำนวณ	ขึ้นกับจำนวนแผนที่อยู่ใน Atlas	ล็อกไว้คงที่ $O(1)$ ใน Working Memory
การใช้ทรัพยากร RAM/CPU	เพิ่มขึ้นตามขนาดข้อมูล Atlas	มีเกณฑ์ควบคุมขีดจำกัดสูงสุด (Memory Bounded)
ความทนทานต่อการขยับ/สะเทือน	ปานกลาง-สูง (พึ่งพา ORB Features)	สูงมาก (VINS มี Online Calibration)

3.2 รายละเอียดชิ้นส่วนฮาร์ดแวร์และสภาพแวดล้อมประมวลผล (Hardware & Software Setup)

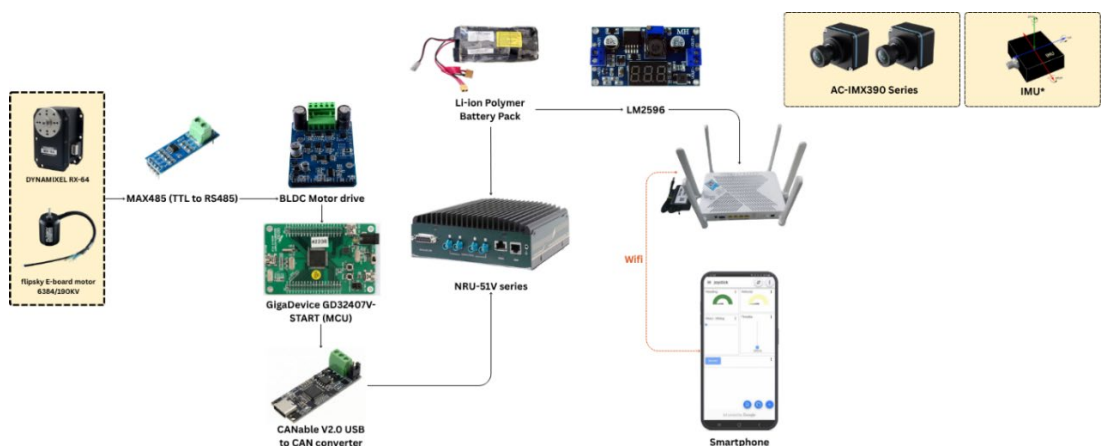
1) หุ่นยนต์ทดสอบเคลื่อนที่อัตโนมัติ (AMR Mobile Robot Testbed)

ระบบได้รับการติดตั้งและทดสอบบนฐานหุ่นยนต์เคลื่อนที่อัตโนมัติแบบล้อขับเคลื่อนสองล้ออิสระ (Differential Drive Mobile Robot) ของบริษัท Gensurv Robotics ซึ่งถูกออกแบบมาเพื่อจำลองพฤติกรรมการทำงานของ AMR ในคลังสินค้าอุตสาหกรรม



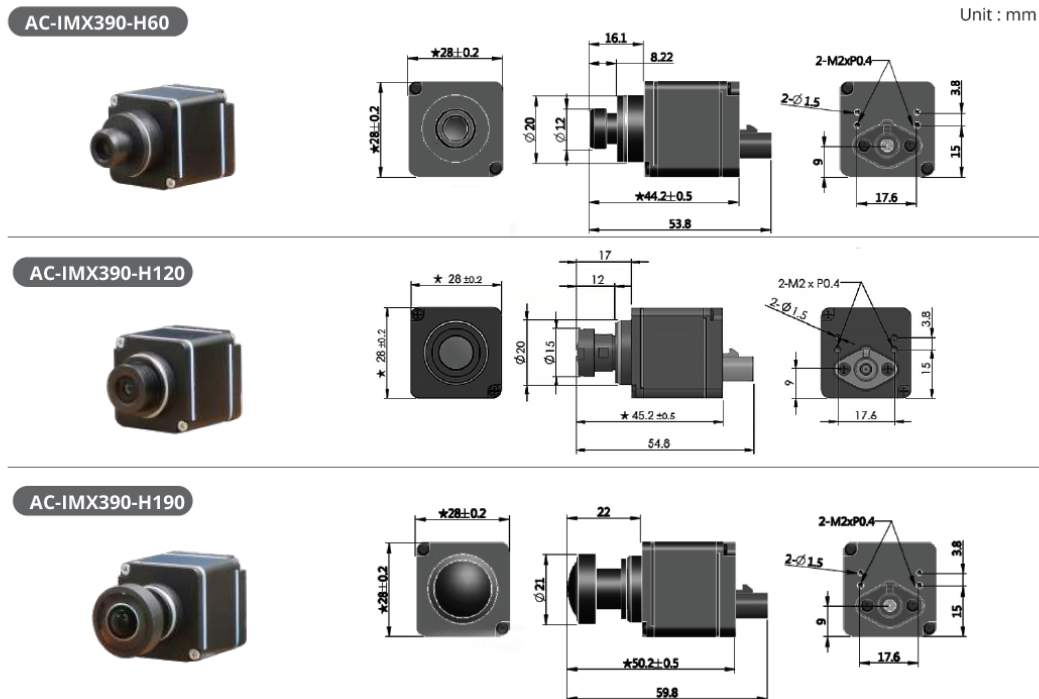
รูปที่ 14 แสดงขนาดของหุ่น AMR จำลองสำหรับการทดสอบระบบนำทาง

- ความเร็วเคลื่อนที่สูงสุด (Max Speed) : กำหนดไว้ที่ 1.5 เมตรต่อวินาทีตามขอบเขตการทดสอบความเสถียรของระบบ SLAM
- จุดเด่น : มีความเสถียรสูงด้านการจัดการทรัพยากรฮาร์ดแวร์ ป้องกันปัญหา RAM เต็มบนบอร์ดประมวลผลขนาดเล็กเมื่อทำงานต่อเนื่องเป็นเวลานาน



รูปที่ 15 แสดงองค์ประกอบทางฮาร์ดแวร์ของหุ่น AMR จำลองสำหรับการทดสอบระบบนำทาง

2) กล้องสเตอริโอเลนส์มุมกว้าง (Stereo Cameras - AC-IMX390-H190):



รูปที่ 16 แสดงขนาดของกล้อง AC-IMX390-H190

ตารางที่ 3 แสดงรายละเอียดของกล้อง AC-IMX390-H190

สเปค	รายละเอียด
Sensor & ISP	Sony IMX390 CMOS sensor and GEO SEMI GW5200 ISP
Resolution & FPS	1920x1080 (2MP) @ 30 FPS
Field of View (HFOV)	186°
Video Output	Uncompressed YUV422 (UYVY)
Dynamic Range	120dB High Dynamic Range (HDR)
Special Features	LED Flicker Mitigation (LFM), 5-axis active alignment
Environmental	IP67 and IP69K waterproof (lens side)
Operating Temperature	-40°C to 85°C
Connector	Male FAKRA connector

3) หน่วยวัดความเฉื่อย (Inertial Measurement Unit - IMU):

- ทำหน้าที่วัดค่าอัตราเร่งเชิงเส้น (Linear Acceleration) และความเร็วเชิงมุม (Angular Velocity) แบบ 6 แกน (6-AXIS) ด้วยความถี่สูง (100 - 200 Hz)
- ข้อมูล IMU จะถูกนำมาทำกระบวนการ IMU Pre-integration เพื่อผสานร่วมกับภาพถ่ายจากกล้อง ช่วยประมาณค่า Odometry ในช่วงที่กล้องเคลื่อนที่เร็ว หรือเกิดการเบลอชั่วขณะ (Motion Blur)

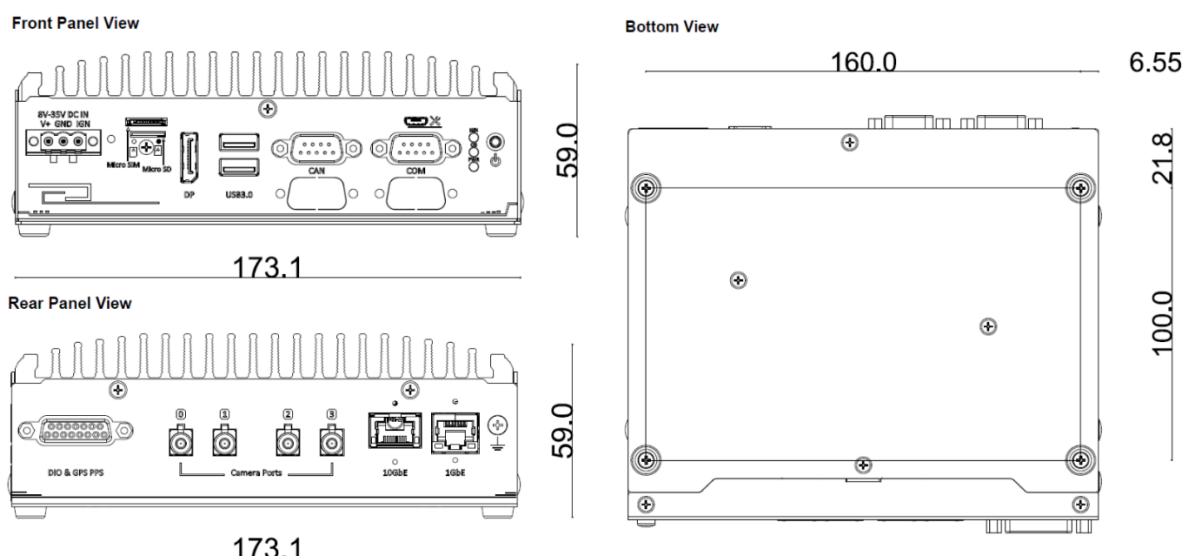
4) หน่วยประมวลผลระดับเอดจ์ (Edge AI Computing Unit): Neosys NRU-51V

(NVIDIA Jetson Xavier NX)

ในการประมวลผลโมเดล Deep Learning (YOLO26) ควบคู่ไปกับระบบ Visual-Inertial SLAM บนหุ่นยนต์ AMR โดยไม่พึ่งพาเครื่องประมวลผลส่วนกลาง โครงการนี้เลือกใช้คอมพิวเตอร์อุตสาหกรรมขนาดเล็ก Neosys NRU-51V ซึ่งเป็นบอร์ดประมวลผลระดับเอดจ์ที่ถูกออกแบบมาเพื่อระบบวิชันและยานยนต์อัตโนมัติโดยเฉพาะ



รูปที่ 17 แสดงรูปของ Neosys NRU-51V (NVIDIA Jetson Xavier NX)



รูปที่ 18 แสดงขนาดของ Neosys NRU-51V (NVIDIA Jetson Xavier NX) ในหน่วยมิลลิเมตร

ตารางที่ 4 แสดงการสรุปข้อมูลจำเพาะทางเทคนิคของฮาร์ดแวร์

อุปกรณ์ / เซนเซอร์	รุ่น / ข้อมูลจำเพาะ	หน้าที่และบทบาทในระบบ
Edge Processing Unit	Neousys NRU-51V (NVIDIA Jetson Xavier NX, 21 TOPS)	ประมวลผลโมเดล YOLO26, ระบบ SLAM และ ROS2 Navigation Stack
Stereo Camera Module	AC-IMX390-H190 (Sony IMX390 Sensor)	บันทึกภาพสเตอริโอ 186° FOV สำหรับตรวจจับวัตถุและสกัดจุดเด่นทางภาพ
Inertial Sensor (IMU)	6-DOF Industrial IMU (\$100-200 \text{ Hz})	วัดค่าอัตราเร่งและความเร็วเชิงมุมเพื่อชดเชยการประเมินตำแหน่ง (VIO)
Mobile Testbed	AMR Platform (Speed 1.5 m/s, Slope 5°)	ฐานหุ่นยนต์สำหรับทดสอบระบบนำทางออนไลน์ในสภาพแวดล้อมจำลอง

3.3 กระบวนการประมวลผลส่วนหน้าและการคัดกรองวัตถุพลวัต (Frontend Processing & Dynamic Masking)

ในสภาวะแวดล้อมคลังสินค้าที่มีการเปลี่ยนแปลงตลอดเวลา (Dynamic Warehouse) อัลกอริทึม SLAM แบบดั้งเดิมจะเกิดความคลาดเคลื่อนสะสม (Drift) หากนำจุดเด่น (Features) บนวัตถุที่กำลังเคลื่อนที่ไปคำนวณระยะทาง โครงการนี้จึงออกแบบกระบวนการประมวลผลส่วนหน้า (Frontend) แบบทำงานร่วมกัน (Shared Frontend) โดยบูรณาการโมเดลปัญญาประดิษฐ์เข้ากับกระบวนการคัดกรองจุดเด่นทางเรขาคณิต เพื่อจำกัดข้อมูลขยะ (Outliers) ก่อนส่งต่อไปให้ระบบส่วนหลัง

1) การตรวจจับสิ่งกีดขวางและพัสดุด้วยโมเดล YOLO26 (Object Detection Module)

เมื่อระบบรับข้อมูลเฟรมภาพ I_t ณ เวลา t จากกล้องสเตอริโอ ภาพจะถูกส่งเข้าสู่โครงข่ายประสาทเทียม YOLO26 เพื่อทำการอนุมาน (Inference) และระบุตำแหน่งของวัตถุพลวัตในระยะประชิด (Close-Proximity Dynamic Objects) เช่น พัสดุ รถเข็น หรือบุคคล ผลลัพธ์จากการตรวจจับจะอยู่ในรูปแบบของเซตของกรอบล้อมรอบ (Bounding Boxes) B ดังสมการ:

$$B = \{B_1, B_2, B_3, \dots, B_K\}$$

โดยที่ B_K คือข้อมูลของวัตถุที่ตรวจพบตัวที่ k ซึ่งประกอบด้วยพารามิเตอร์ดังนี้

$$B_K = \{x_k, y_k, w_k, h_k, c_k, P_k\}$$

โดยที่ :

- (x_k, y_k) คือพิกัดจุดศูนย์กลางของกรอบล้อมรอบในภาพ
- (w_k, h_k) คือความกว้างและความสูงของกรอบล้อมรอบ
- c_k คือประเภทของวัตถุที่จำแนกได้ (Class label)
- P_k คือค่าความมั่นใจในการทำนาย (Confidence score)

เพื่อป้องกันความผิดพลาดจากการผลตรวจจับที่ไม่มีความแม่นยำ (False Positives) ระบบจะทำการกรองข้อมูลโดยกำหนดค่าเกณฑ์ความมั่นใจขั้นต่ำ (Confidence Threshold: τ_{conf}) หาก $P_k < \tau_{conf}$ กรอบล้อมรอบนั้นจะถูกปฏิเสธและไม่นำมาใช้ในกระบวนการถัดไป

2) กลไกคัดกรองจุดเด่นพลวัต (Dynamic Feature Masking Protocol)

ในขณะที่เธรดการประมวลผลปัญญาประดิษฐ์กำลังทำงาน เธรดการติดตามตำแหน่ง (Tracking Thread) ของระบบ SLAM (ทั้งใน ORB-SLAM3 หรือ VINS) จะทำการสกัดจุดเด่นทางเรขาคณิต (เช่น จุดเด่นแบบ ORB หรือ FAST/Harris corners) จากภาพเดียวกัน I_t ให้เซตของจุดเด่นทั้งหมดในเฟรมภาพคือ F :

$$F = \{F_1, F_2, F_3, \dots, F_N\}$$

โดยที่ $f_i = (u_k, v_k)$ คือพิกัดของจุดเด่นตัวที่ i บนระนาบภาพ 2 มิติ เมื่อระบบได้รับข้อมูล Bounding Boxes B จาก YOLO26 แล้ว จะเข้าสู่กระบวนการ Dynamic Feature Masking โดยระบบจะสร้างพื้นที่สกัดกั้น (Mask Region) สำหรับวัตถุพลวัตแต่ละชิ้น จากนั้นอัลกอริทึมจะตรวจสอบเงื่อนไขเชิงพื้นที่ (Spatial condition) ของจุดเด่น f_i ทุกจุด ว่าตกอยู่ภายในขอบเขตของกรอบล้อมรอบพลวัต B_k ใด ๆ หรือไม่ ดังสมการ:

$$\exists B_k \in B : (x_k - \frac{w_k}{2} \leq u_i \leq x_k + \frac{w_k}{2}) \wedge (y_k - \frac{h_k}{2} \leq v_i \leq y_k + \frac{h_k}{2})$$

โดยจุดเด่น $f_i(u_k, v_k)$ จะถูกระบุว่าเป็น จุดเด่นพลวัต (Dynamic Feature / Outlier) หากสอดคล้องกับเงื่อนไขสมการข้างต้นนี้ หากเงื่อนไขดังกล่าวเป็นจริง (True) จุดเด่น f_i จะถูกลบทิ้งออกจากระบบทันที

3) ผลลัพธ์และการส่งต่อข้อมูลสู่กระบวนการระบุตำแหน่ง (Output & Backend

Integration)

ผลลัพธ์จากกลไกการสกัดกั้น จะทำให้ระบบเหลือเพียงเซตของ จุดเด่นสถิต (Static Feature Set: F_{static}) ซึ่งเป็นจุดเด่นที่มาจากโครงสร้างถาวรของอาคารหรือฉากหลัง (Static Background) เท่านั้น:

$$F_{static} = \{f_i \in F | f_i \notin B_k, \forall k\}$$

เซตของจุดเด่น F_{static} นี้จะถูกส่งต่อไปยังส่วนหลัง (Backend) เพื่อใช้ในการจับคู่คุณลักษณะ (Feature Matching) และคำนวณสมการเพื่อลดค่าความคลาดเคลื่อนของการฉายภาพซ้ำ (Reprojection Error Minimization) ทำให้ระบบสามารถประเมินตำแหน่ง (Odometry) แบบ 6-DOF ได้อย่างแม่นยำ ปราศจากการรบกวนของจุดเด่นจากวัตถุพลวัตระยะประชิด ซึ่งเป็นการแก้ปัญหาคความทนทาน (Robustness) อย่างตรงจุด

3.4 กระบวนการสร้างแผนที่และระบุตำแหน่งระยะยาว (Backend Lifelong Mapping Algorithms)

1) การทำงานของไปป์ไลน์ที่ 1 (YOLO26 + ORB-SLAM3 Pipeline)

ไปป์ไลน์สถาปัตยกรรมรูปแบบที่ 1 เป็นระบบการระบุตำแหน่งและสร้างแผนที่ด้วยภาพผสมความเฉื่อยแบบรวมศูนย์ (Tightly-Coupled Visual-Inertial SLAM) โดยประยุกต์ใช้กรอบการทำงาน ORB-SLAM3 ร่วมกับข้อมูลจุดเด่นสถิต (F_{static}) ที่ผ่านการคัดกรองวัตถุพลวัตจาก YOLO26 ในส่วนประมวลผลส่วนหน้า (Shared Frontend) ดังมีรายละเอียดกลไกทางเทคนิคและสมการคณิตศาสตร์ดังต่อไปนี้

- การประเมินตำแหน่งแบบประมาณค่าความน่าจะเป็นสูงสุดหลังปรับปรุง (MAP Estimation Framework)

ORB-SLAM3 ใช้หลักการประมาณค่าความน่าจะเป็นสูงสุดหลังปรับปรุง (Maximum A Posteriori: MAP Estimation) ในการคำนวณหาสถานะของหุ่นยนต์ (Robot State Vector: X_k) ณ คีย์เฟรมที่ k ซึ่งประกอบด้วยพิกัดตำแหน่ง ความเร็ว และค่าเบี่ยงเบนของเซนเซอร์ความเฉื่อย ดังสมการ:

$$X_k = [R_k, p_k, v_k, b_a, b_g]$$

โดยที่ :

- $R_k \in SO(3)$ คือ เมทริกซ์การหมุนตัว (Orientation) ของตัวถึงหุ่นยนต์เทียบกับกรอบอ้างอิงโลก (World Frame)
- $p_k \in \mathbb{R}^3$ คือพิกัดตำแหน่งเชิงพื้นที่ (Position) ของหุ่นยนต์
- $v_k \in \mathbb{R}^3$ คือความเร็วเชิงเส้น (Velocity) ของหุ่นยนต์
- b_a, b_g คือค่าความคลาดเคลื่อนฐาน (Bias) ของเครื่องวัดความเร่ง (Accelerometer) และไจโรสโคป (Gyroscope) ตามลำดับ

ในการคำนวณปรับแต่งโครงข่ายกราฟเฉพาะส่วน (Local Bundle Adjustment) ระบบจะทำการลดค่าฟังก์ชันต้นทุนรวม (Total Cost Function: $E(X)$) ซึ่งเป็นผลรวมของความคลาดเคลื่อนจากการฉายภาพซ้ำของจุดเด่นสถิต (E_{visual}) และความคลาดเคลื่อนตกค้างจากการประมวลผลเซนเซอร์ความเฉื่อย (E_{IMU}) ดังสมการ:

$$\min_x \left(\sum_k \sum_{j \in F_{static}} E_{visual}(X_k, P_j) + \sum_k E_{IMU}(X_k, X_{k+1}) \right)$$

โดยที่ $P_j \in \mathbb{R}^3$ คือ พิกัดสามมิติของจุดเด่นสถิตในแผนที่ (3D Map Landmark)

- สมการความคลาดเคลื่อนตกค้างทางภาพและความเฉื่อย (Reprojection & IMU Residuals)

1. ความคลาดเคลื่อนจากการฉายภาพซ้ำของคุณลักษณะเด่นสถิต (e_{visual}):

จุดเด่น $f_i \in F_{static}$ ที่ผ่านการสกัดกั้นวัตถุพลวัตแล้ว จะถูกวัดค่าความคลาดเคลื่อนระหว่างพิกัดภาพที่สังเกตได้จริง ($x_{k,j}$) กับพิกัดที่ได้จากการฉายจุดสามมิติ P_j ลงบนระนาบกล้อง ดังสมการ:

$$e_{visual} = x_{k,j} - \pi(T_{c,b}T_{w,b_k}^{-1}P_j)$$

โดยที่ :

- $\pi(\cdot)$ คือ ฟังก์ชันการฉายภาพตามแบบจำลองกล้อง (Camera Projection Function)
- $T_{c,b} \in SE(3)$ คือ เมทริกซ์แปลงพิกัดระหว่างกล้องกับตัวถังหุ่นยนต์ (Extrinsic Calibration)
- T_{w,b_k} คือ เมทริกซ์แปลงพิกัดตำแหน่งหุ่นยนต์ ณ เวลา k

2. ความคลาดเคลื่อนตกค้างจากการประมวลผล IMU ล่วงหน้า (e_{IMU}):

เพื่อเชื่อมโยงสถานะระหว่างคีย์เฟรม k และ $k + 1$ โดยไม่ต้องคำนวณสมการเชิงอนุพันธ์ซ้ำซ้อน โดยที่ g คือ แรงโน้มถ่วงของโลก และ Δt คือ ระยะเวลาประมวลผลระหว่างสองคีย์เฟรม นอกจากนี้ระบบยังใช้เทคนิค IMU Pre-integration ซึ่งมีสมการความคลาดเคลื่อนตกค้างของตำแหน่ง (Δp), ความเร็ว (Δv) และทิศทาง (ΔR) ดังนี้:

$$e_{IMU} = \begin{bmatrix} R_k^T \left(p_{k+1} - p_k - v_k \Delta t - \frac{1}{2} g \Delta t^2 \right) - \Delta p_{k,k+1} \\ R_k^T (v_{k+1} - v_k - g \Delta t) - \Delta v_{k,k+1} \\ 2[\Delta t_k^T \cdot (R_k^T R_{k+1})]_{xyz} \end{bmatrix}$$

- การจัดการแผนที่ระยะยาวด้วยระบบแผนที่พหุคูณ (Atlas Multi-Map System)

เพื่อแก้ปัญหาการเสื่อมสภาพของแผนที่ (Map Degradation) ในคลังสินค้าที่มีการย้ายพัสดุและสิ่งกีดขวาง โครงการนี้ใช้สถาปัตยกรรม Atlas ใน ORB-SLAM3 ซึ่งบริหารจัดการเซตของแผนที่ทั้งหมด $\mathcal{M} = \{M_1, M_2, \dots, M_m\}$ โดยแบ่งออกเป็น 2 สถานะ:

1. การสร้างแผนที่ย่อยเมื่อเกิดภาวะสูญเสียการติดตาม (Sub-map Creation on Tracking Loss):

หากหุ่นยนต์เคลื่อนที่ผ่านบริเวณที่สภาพแวดล้อมเปลี่ยนแปลงไปจากเดิมอย่างรุนแรง จนจำนวนจุด Inlier ของ F_{static} ลดลงต่ำกว่าเกณฑ์ความปลอดภัย ($N_{inlier} < \tau_{lost}$) ระบบส่วนหน้าจะระบุสถานะเป็น Tracking Lost ในระบบ SLAM มาตรฐานทั่วไป การทำงานจะล้มเหลว แต่ใน Atlas ระบบจะทำการ พักแผนที่ปัจจุบัน (M_{active}) ไว้เป็น Non-active Map จากนั้นจะทำการเริ่มตั้งต้นระบบใหม่ (Re-initialization) เพื่อสร้าง แผนที่ย่อยชิ้นใหม่ (M_{new}) ขึ้นมาทำหน้าที่เป็น Active Map นำทางต่อทันที ทำให้ระบบไม่ค้างและไม่สูญเสียการทำงาน

2. การตรวจจับสถานที่และการผสานแผนที่แบบไร้รอยต่อ (Place Recognition & Map Merging):

ในขณะที่หุ่นยนต์เคลื่อนที่ไปในคลังสินค้า ระบบ Backend จะนำเวกเตอร์คุณลักษณะทางภาพ (DBow2 Visual Bow Vectors) ของคีย์เฟรมใน M_{active} ไปเปรียบเทียบกับฐานข้อมูลคีย์เฟรมใน Non-active Maps ทั้งหมด หากระบบตรวจพบพื้นที่ที่มีโครงสร้างทางเรขาคณิตตรงกัน (Place Recognition Match) ระบบจะคำนวณเมทริกซ์การแปลงพิกัด $T_{M_a, M_b} \in Sim(3)$ (Similarity Transformation) เพื่อเชื่อมโยงแผนที่ย่อยทั้งสองพื้น และทำการรวมการ Map Merging ผสานจุดสามมิติและคีย์เฟรมเข้าด้วยกัน ก่อนทำ Welded Graph Adjustment เพื่อปรับแก้โครงสร้างแผนที่คลังสินค้าทั้งหมดให้มีความถูกต้องและต่อเนื่องสอดคล้องกับความเป็นจริง

2) การทำงานของไปป์ไลน์ที่ 2 (YOLO26 + VINS + RTAB-Map Pipeline)

ไปป์ไลน์สถาปัตยกรรมรูปแบบที่ 2 เป็นระบบการนำทางที่ใช้แนวคิดการแยกส่วนภาระงาน (Decoupled Architecture) โดยแบ่งหน้าที่ระหว่างการประเมินตำแหน่งความถี่สูงส่วนหน้า (High-frequency Local Odometry) และการสร้างแผนที่พร้อมจัดการหน่วยความจำระยะยาวส่วนหลัง (Long-term Mapping & Memory Management) ออกจากกันอย่างเด็ดขาด เพื่อการทำงานที่เสถียรบนหน่วยประมวลผลระดับเอดจ์ (NRU-51V) ดังมีรายละเอียดกลไกทางเทคนิคและสมการคณิตศาสตร์ดังต่อไปนี้

การประเมินการเคลื่อนที่ส่วนหน้าด้วย VINS (Visual-Inertial Odometry Front-End)

ส่วนประเมินการเคลื่อนที่ที่รับชุดจุดเด่นสถิต (F_{static}) ที่ผ่านการสกัดกั้นวัตถุพลวัตจาก YOLO26 มา ผสานกับสัญญาณความเฉื่อยจาก IMU เพื่อประเมินสถานะ 6-DOF ผ่านกระบวนการปรับแต่งหน้าต่างบัพอัดเคลื่อนที่ (Sliding Window Optimization)

1) เวกเตอร์สถานะและการชดเชยเวลาออนไลน์ (State Vector & Online Temporal Calibration):

จากข้อจำกัดของฮาร์ดแวร์จริงที่มักเกิดความล่าช้าระหว่างสัญญาณภาพและสัญญาณ IMU ระบบจึงได้นำแนวคิดจากงานวิจัย Online Temporal Calibration (Qin & Shen, 2018) มาใช้ โดยเพิ่มตัวแปรค่าความคลาดเคลื่อนเชิงเวลา (t_d) เข้าไปในเวกเตอร์สถานะ χ ในหน้าต่างบัพอัดขนาด N คีย์เฟรมร่วมกับตำแหน่งความลึกตรงข้าม (Inverse Depth: λ) ของจุดเด่นสถิตจำนวน M จุด ดังสมการ:

$$\chi = [x_0, x_1, \dots, x_N, \lambda_a, \dots, \lambda_M, t_d]$$

โดยที่สถานะของหุ่นยนต์ ณ คีย์เฟรมที่ $k(x_k)$ คำนวณจาก:

$$x_k = [p_{b_k}^w, v_{b_k}^w, q_{b_k}^w, b_a, b_g]$$

โดยที่ :

- $p_{b_k}^w, v_{b_k}^w \in \mathbb{R}^3$ คือ คีพตำแหน่งและความเร็วของตัวถังหุ่นยนต์
- $q_{b_k}^w \in S^3$ คือ ควอเทอร์เนียนแสดงทิศทาง (Quaternion Orientation)
- $b_a, b_g \in \mathbb{R}^3$ คือ ค่าความคลาดเคลื่อนฐาน (Bias) ของเครื่องวัดความเร่งและไจโรสโคป

2) สมการปรับแต่งแบบไม่เชิงเส้นในหน้าต่างบัพอัด (Non-linear Sliding Window Optimization):

VINS คำนวณประมาณค่าสถานะ X โดยการลดค่าฟังก์ชันความคลาดเคลื่อนตกค้างรวม (Total Residuals) ซึ่งประกอบด้วยข้อมูลล่วงหน้า (Marginalization Prior), ความคลาดเคลื่อน IMU และความคลาดเคลื่อนทางภาพของคุณลักษณะเด่นสถิติ (ที่นำตัวแปรเวลา t_d มาชดเชยแล้ว) ดังสมการในหัวข้อที่ 2.2.3

3) กลไกขจัดตัวแปร (Marginalization Strategy):

เมื่อมีคีย์เฟรมใหม่เข้ามาในหน้าต่างบัพอัด VINS จะทำการลบคีย์เฟรมที่เก่าที่สุดออก โดยใช้เทคนิค Schur Complement แปลงข้อมูลของคีย์เฟรมเดิมให้กลายเป็นข้อมูลล่วงหน้า (Prior Cost) เพื่อรักษาขนาดเมทริกซ์คำนวณให้คงที่ ส่งผลให้กระบวนการประเมิน VIO Odometry มีความหน่วงต่ำมาก (Low Latency) และส่งออกพิกัดตำแหน่งความถี่สูง (100 Hz) ให้ระบบส่วนหลังได้อย่างเสถียร

การจัดการหน่วยความจำและการสร้างแผนที่ระยะยาว (RTAB-Map Memory Management & Backend)

พิกัด Odometry และข้อมูลภาพสเตอริโอจะถูกส่งต่อเข้าสู่กรอบการทำงาน RTAB-Map ซึ่งทำหน้าที่เป็น Graph SLAM ส่วนหลัง และควบคุมปริมาณการใช้หน่วยความจำ RAM ให้คงที่ตลอดการทำงานระยะยาว (Lifelong Mapping)

ตารางที่ 5 แสดงการเปรียบเทียบกลไกเชิงทฤษฎีของทั้ง 2 ไปป์ไลน์

มิติการทำงานทางเทคนิค	ไปป์ไลน์ที่ 1: YOLO26 + ORB-SLAM3	ไปป์ไลน์ที่ 2: YOLO26 + VINS + RTAB-Map
การคำนวณ Odometry	Tightly-coupled MAP Estimation รวมทั้ง Visual และ IMU	Sliding Window Optimization (VINS Local VIO)
กลไก Lifelong Mapping	Atlas Multi-map System (สร้างและผสานแผนที่ย่อย $Sim(3)$)	Memory Management System (ย้ายโหนดระหว่าง WM และ LTM)
ขอบเขตความซับซ้อนเวลา (STS)	ขึ้นอยู่กับจำนวนคีย์เฟรมและแผนที่ย่อยใน Atlas	จำกัดเพดานสูงสุดไว้คงที่ $O(1)$ ใน Working Memory
การจัดการหน่วยความจำ RAM	ใช้ RAM เพิ่มขึ้นตามขนาดพื้นที่แผนที่สะสม	ล็อกการใช้ RAM คงที่ผ่านเกณฑ์เวลาประมวลผล T_{max}
ความทนทานต่อแรงสั่นสะเทือน	พึ่งพาการสแก็คจุดเด่น ORB และ IMU Pre-integration	สูงมาก (มี Online Temporal & Spatial Calibration ใน VINS)

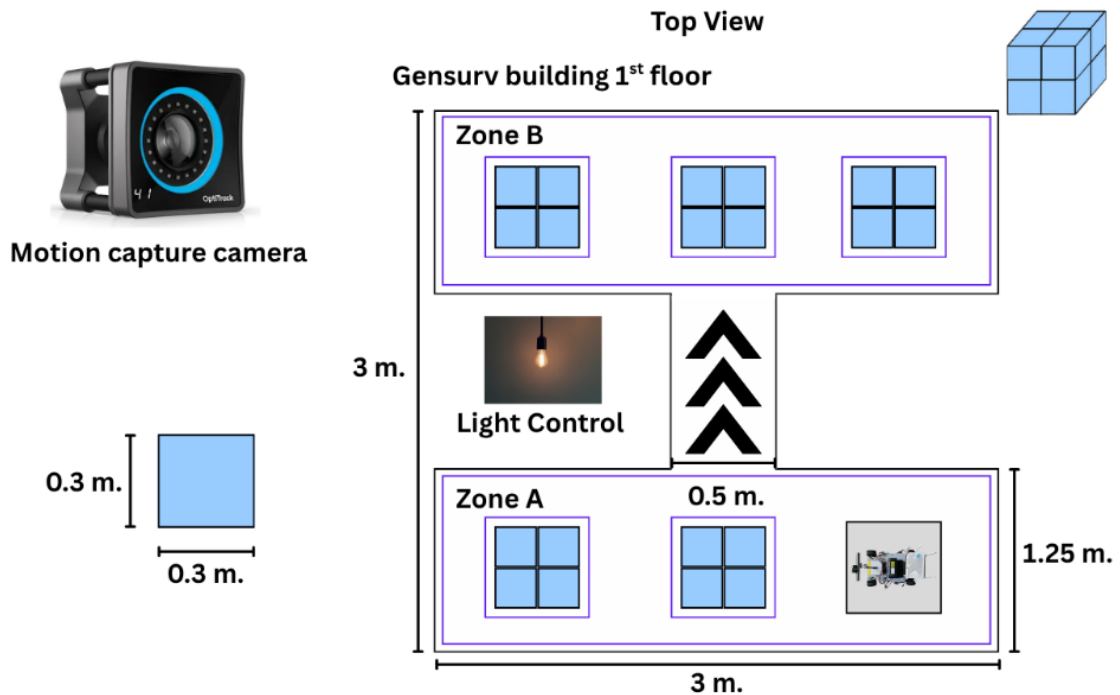
3.5 การออกแบบการทดลองและการเก็บข้อมูล (Experimental Design & Data Collection)

เพื่อประเมินประสิทธิภาพของสถาปัตยกรรมระบบนำทางทั้ง 2 ไปป์ไลน์ โครงการนี้ได้ออกแบบระเบียบวิธีทดสอบเชิงประจักษ์ (Empirical Testing) บนหุ่นยนต์จริง (Online Testing) โดยจำลองสภาพแวดล้อมให้ใกล้เคียงกับคลังสินค้าอุตสาหกรรมที่มีการเคลื่อนย้ายสิ่งของตลอดเวลา (Dynamic Warehouse)

1) การจัดตั้งสนามทดสอบจำลองคลังสินค้า (Simulated Warehouse Environment Setup)

การทดสอบดำเนินการ ณ พื้นที่ชั้น 1 อาคารสถาบันนวัตกรรมหุ่นยนต์และยานยนต์อัตโนมัติ (สนอ.) มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าธนบุรี และพื้นที่จำลองคลังสินค้าของบริษัท Gensurv Robotics โดยมีการจัดเตรียมพื้นที่และระบบอ้างอิงดังต่อไปนี้:

- **โครงสร้างพื้นที่ทดสอบเชิงพื้นที่ (Spatial Layout Configuration)** : พื้นที่ทดสอบถูกแบ่งออกเป็น 2 โซนปฏิบัติการหลัก ได้แก่ Zone A และ Zone B โดยมีระยะห่างและทางเดินเชื่อมต่อกัน (ขนาดพื้นที่อ้างอิงประมาณ 3×3 เมตรต่อโซน พร้อมทางเดินกว้าง 1.25 เมตร) การแบ่งโซนนี้มีวัตถุประสงค์เพื่อจำลองเส้นทางการสัญจร (Pattern Trajectory) ของหุ่นยนต์ AMR ที่ต้องวิ่งไป-กลับระหว่างจุดรับของและจุดส่งของอย่างต่อเนื่อง โดยหุ่นยนต์จะถูกกำหนดความเร็วเชิงเส้น (Linear Velocity) สูงสุดไว้ที่ $v_{\max} = 1.5$ เมตรต่อวินาทีตลอดการทดสอบ
- **ระบบอ้างอิงพิกัดความแม่นยำสูง (Ground Truth Setup via Motion Capture)** : ในการประเมินความคลาดเคลื่อนของระบบ SLAM (เช่น ATE และ RPE) ระบบจำเป็นต้องมีค่าพิกัดอ้างอิงที่แม่นยำในระดับมิลลิเมตร (Ground Truth) โครงการนี้จึงติดตั้งระบบกล้องจับภาพเคลื่อนไหว (Motion Capture Camera System) ไว้รอบพื้นที่ทดสอบ เพื่อติดตามมาร์กเกอร์สะท้อนแสงที่ติดตั้งอยู่บนจุดศูนย์กลางถ่วง (Center of Mass) ของหุ่นยนต์ AMR ด้วยความถี่สูง (>100 Hz)
- **สมการจัดตำแหน่งกรอบอ้างอิงพิกัด (Coordinate Frame Alignment)** : เนื่องจากพิกัดตำแหน่งที่ได้จากระบบ SLAM (P_{slam}) จะอ้างอิงกับจุดเริ่มต้นการเปิดเครื่อง (Local Odometry Frame) ในขณะที่พิกัดจากกล้อง Motion Capture (P_{gt}) จะอ้างอิงกับพิกัดโลกของห้อง (Global World Frame) ดังนั้น ก่อนที่จะนำผลลัพธ์มาคำนวณหาความคลาดเคลื่อน ระบบจะต้องทำการจัดเรียงกรอบพิกัด (Trajectory Alignment) ผ่านสมการการแปลงรูปแบบโครงสร้างแข็ง (Rigid Body Transformation) ดังนี้:



รูปที่ 19 แสดงรูปแผนที่ของสนามทดสอบจำลองในมุมมอง Top view

2) การกำหนดตัวแปรทดลอง (Experimental Variables)

ตัวแปรด้านความพลวัตของพื้นที่ (Dynamic Environment Variable)

- เงื่อนไขการทริกเกอร์ (Trigger Condition): ทุกครั้งที่หุ่นยนต์สัญญาณเปลี่ยนโซน (เช่น เคลื่อนที่จาก Zone A ไปถึง Zone B สำเร็จ) ผู้ทดสอบจะทำการสลับเปลี่ยนตำแหน่งและจำนวนของกองพัสดุในโซนตรงข้าม (Zone A) ใหม่ทั้งหมด ก่อนที่หุ่นยนต์จะวิ่งกลับมา กระบวนการนี้จะบีบบังคับให้ระบบ SLAM ต้องเผชิญกับสภาพแวดล้อมที่เปลี่ยนแปลงไปจากแผนที่เดิม (Map Obsolescence) อยู่ตลอดเวลา เพื่อพิสูจน์ว่าระบบ Atlas ของ ORB-SLAM3 หรือกลไก Memory Management ของ RTAB-Map สามารถจัดการกับความขัดแย้งของข้อมูลเชิงพื้นที่นี้ได้โดยไม่เกิดภาวะ Tracking Lost

ตัวแปรด้านสถานะแสงสว่าง (Illumination Condition Variable)

การทดลองจะดำเนินการเก็บข้อมูลตามเส้นทางสัญญาณเดิมใน 3 ช่วงเวลาที่แตกต่างกันของวัน เพื่อเป็นตัวแทนของระดับความสว่างและทิศทางของแสงตามสมมุติฐานที่คิดไว้ได้แก่:

- (ช่วงเช้า - 08:00 น.): สถานะแสงธรรมชาติทำมุมเงย เกิดเงาทอดทับ (Casting Shadows) บนพื้นที่ทดสอบ
- (ช่วงกลางวัน - 12:00 น.): สถานะแสงสว่างสูงสุด อาจก่อให้เกิดปัญหาความเปรียบต่างสูง (High Contrast / Overexposure) ในบางบริเวณ

- (ช่วงเย็น/ค่ำ - 18:00 น.): สภาวะแสงน้อย (Low-light condition) ซึ่งต้องพึ่งพาแสงสว่างจากหลอดไฟประดิษฐ์ภายในอาคารเป็นหลัก ส่งผลให้เกิดสัญญาณรบกวนภาพ (Image Noise) สูงขึ้น

การกำหนดตัวแปรแสงนี้ จะช่วยยืนยันประสิทธิภาพของฮาร์ดแวร์เซนเซอร์กล้อง (Sony IMX390 HDR) และประเมินความทนทานของกรอบการทำงาน SLAM ทั้งสองรูปแบบ ว่าสามารถรักษาจำนวนของจุดเด่น (Inlier Features) ให้เพียงพอต่อการคำนวณ Odometry ในสภาวะแสงที่แปรผันได้ดีเพียงใด

3) ขั้นตอนและกระบวนการทดสอบระบบแบบออนไลน์และออฟไลน์ (Online & Offline Integrated Testing Procedure)

การประเมินผลสถาปัตยกรรมระบบนำทางทั้ง 2 รูปแบบ จะถูกดำเนินการทดสอบแบบเวลาจริงบนฮาร์ดแวร์หุ่นยนต์ (Online Real-world Testing) ผ่านกรอบการทำงานระบบปฏิบัติการหุ่นยนต์ (ROS 2) โดยมีขั้นตอนการดำเนินงานมาตรฐานดังต่อไปนี้:

ขั้นตอนที่ 1: การตั้งต้นและซิงโครไนซ์ระบบ (Initialization and Time Synchronization)

- 1) เปิดการทำงานของหน่วยประมวลผลระดับเอดจ์ (NRU-51V) บนหุ่นยนต์ AMR และเริ่มต้นระบบเซนเซอร์กล้องสเตอริโอและหน่วยวัดความเฉื่อย (IMU)
- 2) เปิดการทำงานของระบบกล้อง Motion Capture (MoCap) และดำเนินการสอบเทียบพื้นที่ (Room Calibration)
- 3) การซิงโครไนซ์เวลา (Time Synchronization): เนื่องจากความคลาดเคลื่อนของเวลาเพียงเสี้ยววินาทีอาจทำให้ผลการคำนวณ ATE/RPE ผิดพลาด ระบบ ROS 2 บนหุ่นยนต์และเซิร์ฟเวอร์ MoCap จะถูกซิงโครไนซ์เวลาให้ตรงกันผ่านโปรโตคอล PTP (Precision Time Protocol) หรือ Chrony (NTP) เพื่อให้ฐานเวลาอ้างอิง ตรงกันในระดับไมโครวินาที

ขั้นตอนที่ 2: การดำเนินการเคลื่อนที่และการปรับสภาพแวดล้อม (Execution and Environment Alteration)

- 1) กำหนดให้หุ่นยนต์ AMR เริ่มต้นการเคลื่อนที่จากจุดกำเนิดใน Zone A มุ่งหน้าไปยังเป้าหมายใน Zone B ด้วยความเร็วเดินทางเชิงเส้น (Linear Velocity) ที่ควบคุมไว้ไม่เกิน 1.5 เมตรต่อวินาที ผ่านแพ็คเกจการนำทาง (Nav2)
- 2) เมื่อหุ่นยนต์เคลื่อนที่ถึงจุดหมายใน Zone B ผู้ทดสอบจะทำการทริกเกอร์ ตัวแปรความพลวัต (Dynamic Variable) ตามที่กำหนดไว้ในหัวข้อ 3.5.2 โดยทำการสุ่มวางและเปลี่ยนตำแหน่งกองพัสดุ (\$k\$ กอง) ในพื้นที่ Zone A

- 3) ตั้งให้หุ่นยนต์เคลื่อนที่กลับไปยัง Zone A และทำการปรับเปลี่ยนพัสต์ใน Zone B สลับกันไปมาอย่างต่อเนื่อง กระบวนการนี้จะดำเนินไปเป็นระยะเวลา 5 นาทีต่อ 1 รอบการทดสอบ เพื่อจำลองภาระการทำงานระยะยาว (Lifelong Mapping Task)

ขั้นตอนที่ 3: การบันทึกข้อมูลสถานะของระบบ (Data Logging Protocol) :

ในระหว่างที่หุ่นยนต์กำลังปฏิบัติงาน ระบบจะทำการบันทึกข้อมูล (Logging) ผ่านเครื่องมือ rosbag2 ของ ROS 2 เพื่อเก็บรวบรวมสถานะต่างๆ ไว้สำหรับนำไปคำนวณตัวชี้วัดในภายหลัง โดยข้อมูลที่เก็บมีดังนี้

- พิกัดตำแหน่ง 6-DOF ที่ประเมินได้จากระบบ SLAM (ไปป์ไลน์ที่ 1 หรือ 2) ณ เวลา t
- พิกัดอ้างอิงความแม่นยำสูงจากระบบ Motion Capture ณ เวลา t
- ข้อมูลกรอบล้อมรอบวัตถุพลวัต (Bounding Boxes) และค่าความเชื่อมั่นที่ตรวจจับได้จาก YOLO26
- ระยะเวลาความหน่วง (Latency) ในการประมวลผลไปป์ไลน์ทั้งหมดต่อ 1 เฟรมภาพ

ขั้นตอนที่ 4: การทำซ้ำตามปัจจัยควบคุม (Iteration across Experimental Conditions):

กระบวนการในขั้นตอนที่ 1 ถึง 3 จะถูกทำซ้ำ (Iteration) เพื่อเก็บข้อมูลให้ครบถ้วนตามเมทริกซ์การทดลอง (Experimental Matrix) ซึ่งประกอบด้วย:

- การทดสอบไปป์ไลน์ที่ 1 (YOLO26 + ORB-SLAM3) และ ไปป์ไลน์ที่ 2 (YOLO26 + VINS + RTAB-Map)
- การทดสอบภายใต้สภาวะแสง 3 ระดับ ตามที่กล่าวไว้ในหัวข้อ 3.5.2

โดยข้อมูลที่ถูกบันทึกในรูปแบบไฟล์นามสกุล .db3 (ROS 2 Bag format) ทั้งหมด จะถูกนำไปสกัดพิกัด (Trajectory Extraction) และจัดเรียงเวลา (Timestamp Matching) ก่อนนำเข้าสู่กระบวนการประเมินผลผ่านสคริปต์ประเมินมาตรฐาน (เช่น evo_traj หรือ TUM RGB-D Benchmark Script) ในขั้นตอนการวิเคราะห์ตัวชี้วัดประสิทธิภาพต่อไป

3.6 ตัวชี้วัดประสิทธิภาพและการประเมินผล (Evaluation Metrics)

1) ตัวชี้วัดประสิทธิภาพโมดูลตรวจจับวัตถุ (Object Detection Metrics):

- **ค่าพื้นที่ทับซ้อน (Intersection over Union: IoU) :** โดยกำหนดเกณฑ์การประเมินเบื้องต้นที่ IoU มากกว่า 0.5 จึงจะถือว่าเป็น True Positive
- **Precision :** สัดส่วนการทำนายวัตถุพลวัตที่ถูกต้องเทียบกับการทำนายทั้งหมด
- **Recall :** สัดส่วนของวัตถุพลวัตที่ระบบตรวจพบ เทียบกับจำนวนวัตถุพลวัตที่มีอยู่จริงทั้งหมด

- ค่าคะแนนเอฟวัน (F1 Score) : ใช้ประเมินความสมดุลระหว่าง Precision และ Recall
- ความแม่นยำเฉลี่ย (Average Precision: AP) : คำนวณจากพื้นที่ใต้กราฟ (Area Under Curve) ของ Precision-Recall Curve เพื่อสะท้อนความสามารถรวมของโมเดล

2) ตัวชี้วัดประสิทธิภาพระบบ Visual SLAM (Localization & Mapping Metrics):

- ค่าความคลาดเคลื่อนเส้นทางสัมบูรณ์ (Absolute Trajectory Error: ATE) : ATE ใช้สำหรับประเมิน "ความสอดคล้องเชิงโครงสร้างระดับโลก (Global Consistency)" ของเส้นทางทั้งหมด เหมาะสำหรับการวัดผลการจัดการแผนที่ระยะยาว (Lifelong Mapping) ค่าความคลาดเคลื่อน ณ เฟรมที่สนใจ
- ค่าความคลาดเคลื่อนพิกัดสัมพัทธ์ (Relative Pose Error: RPE) : RPE ใช้สำหรับประเมิน "ความคลาดเคลื่อนสะสมระดับท้องถิ่น (Local Drift)" ที่เกิดขึ้นจากการประเมิน Odometry ในช่วงเวลาหรือระยะทางที่กำหนด ซึ่งสะท้อนถึงเสถียรภาพของ VINS หรือ Tracking Thread

3) ตัวชี้วัดประสิทธิภาพการประมวลผลตามเวลาจริง (Computational Real-Time

Metrics):

- ความหน่วงของระบบ (System Latency) : วัดระยะเวลาดังแต่กล้องจับภาพจนถึงกระบวนการที่ระบบ SLAM ส่งออกพิกัด Odometry สำเร็จ เพื่อให้ระบบรอดพ้นจากปัญหาคอขวด (Bottleneck) ความหน่วงรวมจะต้องน้อยกว่าระยะห่างระหว่างเฟรมภาพของกล้อง
- การใช้ทรัพยากรหน่วยความจำ (Memory Bounded Monitoring) : โดยเฉพาะสำหรับการทดสอบไปป์ไลน์ที่ 2 (RTAB-Map) ระบบจะทำการบันทึกค่าการกินพื้นที่ RAM (Memory Footprint) และเวลาที่ใช้ในการค้นหา Loop Closure ตลอดระยะเวลา 5 นาที เพื่อพิสูจน์สมมติฐานทางคณิตศาสตร์ที่ว่าเวลาการคำนวณถูกบล็อกไว้ที่ $O(1)$ Time Complexity อย่างแท้จริง

เอกสารอ้างอิง

- [1] Cadena, C., Carlone, L., Carrillo, H., Latif, Y., Scaramuzza, D., Neira, J., Reid, I., & Leonard, J. J. (2016). "Past, Present, and Future of Simultaneous Localization and Mapping: Toward the Robust-Perception Age." *IEEE Transactions on Robotics*, 32(6), 1309-1332.
- [2] Saputra, M. R. U., Markham, A., & Trigoni, N. (2018). "Visual SLAM and Structure from Motion in Dynamic Environments: A Survey." *ACM Computing Surveys (CSUR)*, 51(2), 1-36.
- [3] Kanso, H., Singh, A., El Zarif, E., Almohammed, N., Mounsef, J., Maalouf, N., & Arain, B. (2025). "Semantic SLAM: A comprehensive survey of methods and applications." *Intelligent Systems with Applications*, 28, 200591.
- [4] Keith, R., & La, H. M. (2024). "Review of Autonomous Mobile Robots for the Warehouse Environment."
- [5] Chen, X., Li, S., Ren, S., & Wang, J. (2025). "RY-SLAM: A robust YOLO-based real-time visual SLAM solution for dynamic environments." *2025 International Conference on Virtual Reality and Visualization (ICVRV)*. IEEE.
- [6] Campos, C., Elvira, R., Gómez Rodríguez, J. J., Montiel, J. M. M., & Tardós, J. D. (2021). "ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial, and Multi-Map SLAM." *IEEE Transactions on Robotics*, 37(6), 1874-1890.
- [7] Qin, T., Li, P., & Shen, S. (2018). "VINS-Mono: A Robust and Versatile Monocular Visual-Inertial State Estimator." *IEEE Transactions on Robotics*, 34(4), 1004-1020.
- [8] Qin, T., & Shen, S. (2018). "Online Temporal Calibration for Monocular Visual-Inertial Systems." *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 3662-3669.
- [9] Labbé, M., & Michaud, F. (2019). "RTAB-Map as an open-source lidar and visual simultaneous localization and mapping library for large-scale and long-term online operation." *Journal of Field Robotics*, 36(2), 416-446.